



Universidad de Ciencias Aplicadas (UPC)

ALDIMI-PREDICT

Machine Learning - 1ACC0057

Profesor: Robert Ernesto Zubieta Cardenas

Integrantes

Apellidos y Nombres	Código	Roles
Abanto Davila, Alvaro Jair Moises	U202313540	ML Engineer
Ballon Villar, Diego Eduardo	U201520327	MLOps / Developer
Chipoco Mejia Jorge Piero Jesus	U202210236	Data Architect
Vasquez Trujillano, Edson Fabrizio	U202316342	Data Scientist

Índice

1. Resumen Ejecutivo.....	3
1.1. Descripción de la problemática abordada por ALDIMI.....	3
2. Marco Metodológico: CRISP-DM Aplicado.....	4
2.1. Business Understanding.....	6
2.1.1. Objetivos.....	6
2.1.2. Dataset.....	6
2.1.3. Definición de las variables objetivo.....	7
2.1.4. Criterios de éxito técnico.....	8
2.2. Data Understanding & Exploratory Data Analysis (EDA).....	8
2.2.1. Análisis de los conjunto de datos.....	8
2.2.2. Análisis de correlaciones y distribuciones.....	10
2.2.3. Identificación de valores atípicos y datos faltantes.....	16
3. Data Preparación & Feature Engineering:.....	20
3.1. Técnicas de normalización, codificación (encoding) y manejo de desbalance de clases (SMOTE, etc.).....	20
3.2. Creación de nuevas variables (feature selección) que mejoren el poder predictivo del modelo.....	24
4. Modeling:.....	28
4.1. Comparativa de algoritmos evaluados.....	28
4.2. Justificación del modelo seleccionado.....	37
5. Implementación y Despliegue (MLOps).....	41
5.1. Arquitectura del Modelo: Descripción del pipeline de datos desde la base de datos común.....	41
Predicción de Demanda y Logística.....	41
5.2. Dashboard de Visualización: Capturas y explicación de la herramienta de visualización (Streamlit/PowerBI) diseñada para que los directivos de ALDIMI tomen decisiones basadas en los datos.....	44
6. Validación cruzada y ajuste de hiper parámetros.....	55
7. Análisis de Impacto ODS y Ética de Datos.....	67
8. Confluencia con el Ecosistema "ALDIMI Core AI"	68
9. Anexos.....	69
9.1. Enlace al repositorio (GitHub/GitLab).....	69
9.2. Diccionario de datos inicial.....	69

1. Resumen Ejecutivo

1.1. Descripción de la problemática abordada por ALDIMI

Desde su constitución en 2004, la asociación ALDIMI ha consolidado un modelo de atención integral gratuito dirigido a menores con patologías oncológicas en situación de vulnerabilidad extrema. La operatividad de la organización se centra en mitigar las barreras de acceso a la salud para familias provenientes de regiones descentralizadas, proporcionando soporte logístico, alimentario y emocional en la capital.

Sin embargo, el tránsito hacia el modelo "ALDIMI 2.0", el cual proyecta duplicar la capacidad instalada para albergar de 50 a 100 familias, ha evidenciado una crisis de escalabilidad derivada de la dependencia de procesos analógicos que resultan insuficientes ante el incremento de la demanda operativa. La problemática central se divide en tres ejes críticos:

- **Incapacidad de priorizar:** La organización carece de un sistema para la clasificación de pacientes oncológicos según la gravedad de su caso. Esta limitación impide que el equipo actúe de manera proactiva ante pacientes con mayor probabilidad de recaída o deterioro agudo, incrementando la vulnerabilidad de los menores albergados ante crisis de salud.
- **Demanda de alimentos:** La organización recibe donaciones constantemente; sin embargo, no posee un sistema que regule el stock de alimentos. Esto puede causar desabastecimiento, lo cual impedirá brindar la adecuada alimentación a los pacientes.
- **Merma de Recursos:** Existe una problemática aguda de desperdicio y merma de alimentos que caducan por un mal manejo de la rotación, impidiendo que los recursos perecederos lleguen a los beneficiarios antes de su vencimiento.

Ante este escenario, surge el proyecto ALDIMI-Predict. La intervención mediante Machine Learning es vital para garantizar el Derecho a la Salud (ODS 3) y Reducir las Desigualdades (ODS 10). El proyecto se enfoca en el desarrollo de un motor analítico centralizado que procesa datos para generar alertas preventivas en dos frentes:

- **Predicción de Demanda y Logística:** Optimización del inventario para asegurar el abastecimiento de alimentos.
- **Clasificación de Riesgo de Salud:** Clasificación de pacientes con bajo, medio y alto riesgo de complicaciones clínicas.

2. Marco Metodológico: CRISP-DM Aplicado

Plan de trabajo:

[illegible]

Selección de algoritmos baseline				Selección de algoritmos baseline										
Entrenamiento modelo baseline						Entrenamiento modelo baseline								
Dashboard preliminar							Dashboard preliminar							
— Exámenes Parciales — (Semana 8)								— Exámenes Parciales — (Semana 8)						
FASE 4 (Avanzada) — Modelado Avanzado														
XGBoost / Random Forest									XGBoost / Random Forest					
Series temporales (7 y 14 días)									Series temporales (7 y 14 días)					
Ajuste de hiperparámetros									Ajuste de hiperparámetros					
Validación cruzada (k-fold, k=5)									Validación cruzada (k-fold, k=5)					
Integración con base de datos común									Integración con base de datos común					
FASE 5 — Evaluación														
ROC-AUC, RMSE, MAE, F1-Score										ROC-AUC, RMSE, MAE, F1-Score				
Matriz de confusión										Matriz de confusión				
Interpretabilidad (SHAP / importancias)										Interpretabilidad (SHAP / importancias)				
FASE 6 — MLOps & Despliegue														
API REST (FastAPI / Flask)										API REST (FastAPI / Flask)				
Serialización del modelo (joblib)										Serialización del modelo (joblib)				
Dashboard final en Streamlit											Dashboard final en Streamlit			
Monitoreo de drift											Monitoreo de drift			
Documentación técnica y manual											Documentación técnica y manual			
Video demo y sustentación final												Video demo y sustentación final		
Leyenda:	Business Und.	Data EDA	Data Prep.	Modelado	Evaluación	MLOps	Integración	Hito ♦						

2.1. Business Understanding

2.1.1. Objetivos

Para abordar la problemática, se han definido dos objetivos para cada frente:

- Desarrollar modelos de series temporales para predecir el stock crítico de alimentos con horizontes de 7 y 14 días, garantizando el abastecimiento oportuno y reduciendo la merma con el estado del alimento.
- Implementar algoritmo de clasificación supervisada para identificar tempranamente a pacientes oncológicos con alta probabilidad de complicaciones clínicas, permitiendo intervenciones preventivas basadas en datos.

2.1.2. Dataset

- Predicción de Demanda y Logística:

Este dataset constituye el inventario para asegurar el abastecimiento de productos (perecederos y no perecederos) mediante el uso del dataset real Corporación Favorita Grocery Sales Forecasting.

- Especificaciones del Dataset: Cuenta con más de 125 millones de registros históricos de ventas en su conjunto de entrenamiento, detallados por fecha, tienda y artículo.
- Valor Logístico: Incluye el seguimiento histórico de promociones, volumen de ventas y clasificación de artículos (ej. perecederos). Esto permite al motor de ALDIMI analizar el comportamiento de compra histórico, anticipar picos de demanda y planificar la reposición exacta para evitar desabastecimientos en estanterías.
- Variables de Contexto: El modelo integrará datos macroeconómicos (histórico del precio diario del petróleo, fuertemente ligado a la economía local del dataset), calendario de feriados/eventos nacionales y locales, y metadatos de las tiendas (ubicación, ciudad y "clúster") para ajustar el abastecimiento a la realidad de cada zona.
- Acceso:
<https://www.kaggle.com/competitions/favorita-grocery-sales-forecasting/data>

- Clasificación de Riesgo de Salud:

Este dataset constituye el motor para la identificación temprana de complicaciones oncológicas basándose en un volumen masivo de datos clínicos.

- Modelos de Clasificación: Permite el desarrollo de algoritmos para clasificar el "Nivel de Prioridad de Atención" (Bajo/Medio/Alto) basado en variables de agudeza y riesgo.
- Variables de Estilo de Vida: Contiene información sobre la exposición al aire contaminado, el uso de alcohol y la tendencia a fumar de los pacientes.

- Detalle Clínico: Proporciona datos sobre el tipo de cáncer, la etapa del diagnóstico (0 a IV), la puntuación de severidad y los años de supervivencia tras el diagnóstico. Volumen: Incluye microdatos de 50,000 pacientes reales diagnosticados globalmente entre los años 2015 y 2024.
- Acceso: <https://www.kaggle.com/datasets/zahidmughal2343/global-cancer-patients-2015-2024>

2.1.3. Definición de las variables objetivo

Para que el motor analítico de ALDIMI-Predict sea funcional, se han definido variables objetivo específicas para cada frente.

- Predicción de Demanda y Logística:

En este frente, el objetivo es anticipar la demanda de suministros (alimentos mapeados como product_id) para optimizar el inventario perecedero.

- Variable Objetivo (y):
 - demand7: Proyecta la cantidad acumulada necesaria para cubrir la demanda en los 7 días posteriores a la fecha ingresada (dt), calculada como:
$$demand7_t = \sum_{i=1}^7 unitsales_{t+i}.$$
 - demand14: Proyecta la cantidad acumulada necesaria para cubrir la demanda en los 14 días posteriores a la fecha ingresada (dt), calculada como:
$$demand14_t = \sum_{i=1}^{14} unitsales_{t+i}.$$
 - perece: Proyecta el estado del alimento que perece con mayor frecuencia.
- Naturaleza: Variable continua (basada en el campo float64 de unit_sales).
- Justificación: Predecir el volumen de stock mínimo necesario que permita a la administración gestionar donaciones y compras antes de llegar al desabastecimiento. Error máximo: Se establece un error máximo permitido del 8%, dado que el impacto de la ruptura de stock en suministros críticos es alto.

- Clasificación de Riesgo de Salud

El objetivo es identificar de forma automatizada el nivel de prioridad de atención del paciente en base a la severidad de su caso para priorizar recursos de monitoreo y tratamiento.

- Variable Objetivo (y):
 - Prioridad: Una variable categórica que clasifica la prioridad de atención mediante las clases: Bajo, Medio y Alto.
- Naturaleza: Variable categórica.

- Justificación: Esta variable sintetiza indicadores clínicos relacionados al cáncer diagnosticado e indicadores sociales relacionados al estilo de vida del paciente para clasificar la prioridad de la atención médica.
- Error máximo: Consideramos un error máximo permitido del 15% puesto que es fundamental brindar atención a los casos más severos.

2.1.4. Criterios de éxito técnico

- Motor de Predicción de Demanda y Logística

El éxito técnico en este eje se mide por la capacidad de evitar el desabastecimiento sin generar sobrestock de productos perecederos.

Precisión de Pronóstico (Error Máximo): El modelo debe alcanzar un **MAPE (Mean Absolute Percentage Error) < 20%** para los horizontes de 7 y 14 días. Este límite es vital para mitigar el impacto de falta de medicinas críticas

- Motor de Clasificación de Riesgo de Salud

En salud oncológica, el éxito técnico se prioriza en la Sensibilidad (Recall) para no omitir a ningún paciente en riesgo.

Discriminación Clínica (ROC-AUC): El modelo de clasificación supervisada debe alcanzar un **ROC-AUC > 0.85** (tomando como referencia modelos de mortalidad oncológica en MIMIC-IV). Un valor cercano a 1.0 indicaría una capacidad excepcional para distinguir entre niveles de prioridad.

2.2. Data Understanding & Exploratory Data Analysis (EDA)

2.2.1. Análisis de los conjunto de datos

- Predicción de Demanda y Logística:

DESCRIPCIÓN GENERAL DEL DATASET

El dataset Corporación Favorita Grocery Sales Forecasting utilizado para este proyecto contiene aproximadamente 125,497,000 registros de entrenamiento y 3,370,000 de evaluación, cubriendo más de 4 años y medio de operación (del 1 de enero de 2013 al 31 de agosto de 2017), con alrededor de 4,100 SKUs (que incluyen familias de productos perecederos y no perecederos) distribuidos en 54 tiendas de 22 ciudades en Ecuador. Cada registro corresponde a una combinación única de producto, tienda y fecha (granularidad diaria), enriquecida con datos relacionales que incluyen volumen de ventas unitarias, indicadores de promociones activas, metadatos de los artículos y tiendas, calendarios de festivos/eventos (locales y nacionales), flujo de transacciones diarias y variables macroeconómicas como el precio diario del petróleo.

Figura 1

Descripción de todas la variables del dataset Corporación Favorita Grocery Sales Forecasting


```

Tipos de datos en train:
id                int64
date              datetime64[ns]
store_nbr         int8
item_nbr          int32
unit_sales        float64
onpromotion       float64
family            object
perishable        int64
city              object
state             object
type              object

```

Estadísticas descriptivas:

Figura 2

Estadísticas descriptivas del dataset Corporación Favorita Grocery Sales Forecasting

```

Estadísticas descriptivas de variables numericas en train:

```

	unit_sales	onpromotion	perishable
count	3000000.000	0.0	3000000.000
mean	8.481	NaN	0.219
std	18.654	NaN	0.414
min	0.000	NaN	0.000
25%	2.000	NaN	0.000
50%	4.000	NaN	0.000
75%	9.000	NaN	0.000
max	7001.000	NaN	1.000

Figura 3

Tipos de datos del dataset Corporación Favorita Grocery Sales Forecasting

```

Tipos de datos en train:
id                int64
date              datetime64[ns]
store_nbr         int8
item_nbr          int32
unit_sales        float64
onpromotion       float64
family            object
perishable        int64
city              object
state             object
type              object

```

- **Clasificación de Riesgo de Salud**

DESCRIPCIÓN GENERAL DEL DATASET

Dimensiones: 50,000 filas × 15 columnas

Pacientes únicos: 50,000

Rango de edades: 20 a 90 años

Desde: 2015

Hasta: 2024

Figura 4

Descripción de todas la variables del dataset *global_cancer_patients_2015_2024*

TIPOS DE DATOS POR COLUMNA:				
	Tipo	Valores únicos	Nulos	% Nulos
Patient_ID	object	50000	0	0.0
Age	int64	70	0	0.0
Gender	object	3	0	0.0
Country_Region	object	10	0	0.0
Year	int64	10	0	0.0
Genetic_Risk	float64	101	0	0.0
Air_Pollution	float64	101	0	0.0
Alcohol_Use	float64	101	0	0.0
Smoking	float64	101	0	0.0
Obesity_Level	float64	101	0	0.0
Cancer_Type	object	8	0	0.0
Cancer_Stage	object	5	0	0.0
Treatment_Cost_USD	float64	49861	0	0.0
Survival_Years	float64	101	0	0.0
Target_Severity_Score	float64	744	0	0.0

Estadísticas descriptivas:

Figura 5

Estadísticas descriptivas del dataset *global_cancer_patients_2015_2024*

	count	mean	std	min	25%	50%	75%	max	mediana
Age	50000.0	54.4215	20.2245	20.00	37.000	54.00	72.00	89.00	54.00
Genetic_Risk	50000.0	5.0017	2.8858	0.00	2.500	5.00	7.50	10.00	5.00
Air_Pollution	50000.0	5.0101	2.8884	0.00	2.500	5.00	7.50	10.00	5.00
Alcohol_Use	50000.0	5.0109	2.8888	0.00	2.500	5.00	7.50	10.00	5.00
Smoking	50000.0	4.9898	2.8816	0.00	2.500	5.00	7.50	10.00	5.00
Obesity_Level	50000.0	4.9912	2.8945	0.00	2.500	5.00	7.50	10.00	5.00
Treatment_Cost_USD	50000.0	52467.2982	27363.2294	5000.05	28686.225	52474.31	76232.72	99999.84	52474.31
Survival_Years	50000.0	5.0065	2.8833	0.00	2.500	5.00	7.50	10.00	5.00
Target_Severity_Score	50000.0	4.9512	1.1997	0.90	4.120	4.95	5.78	9.16	4.95

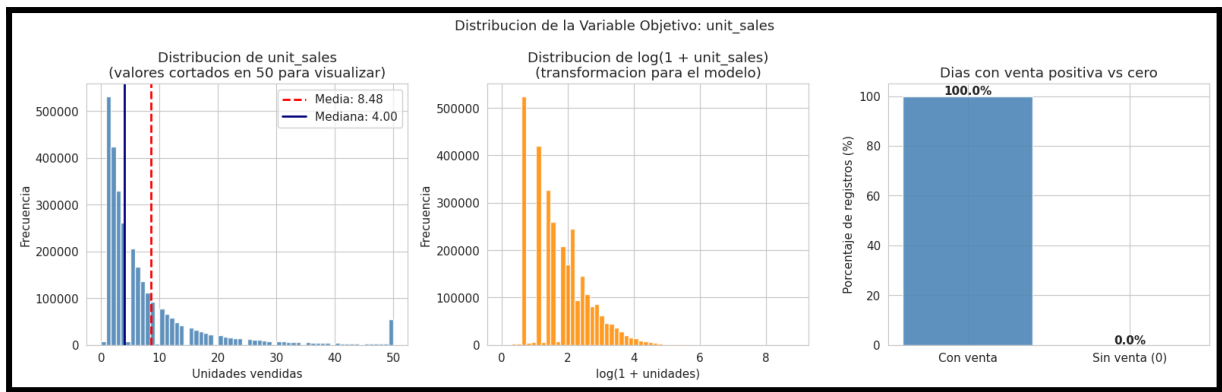
2.2.2. Análisis de correlaciones y distribuciones

- Predicción de Demanda y Logística:

Análisis de Distribución:

Figura 6

Análisis estadístico de la distribución de la variable *sale_amount* de *Corporación Favorita Grocery Sales Forecasting*

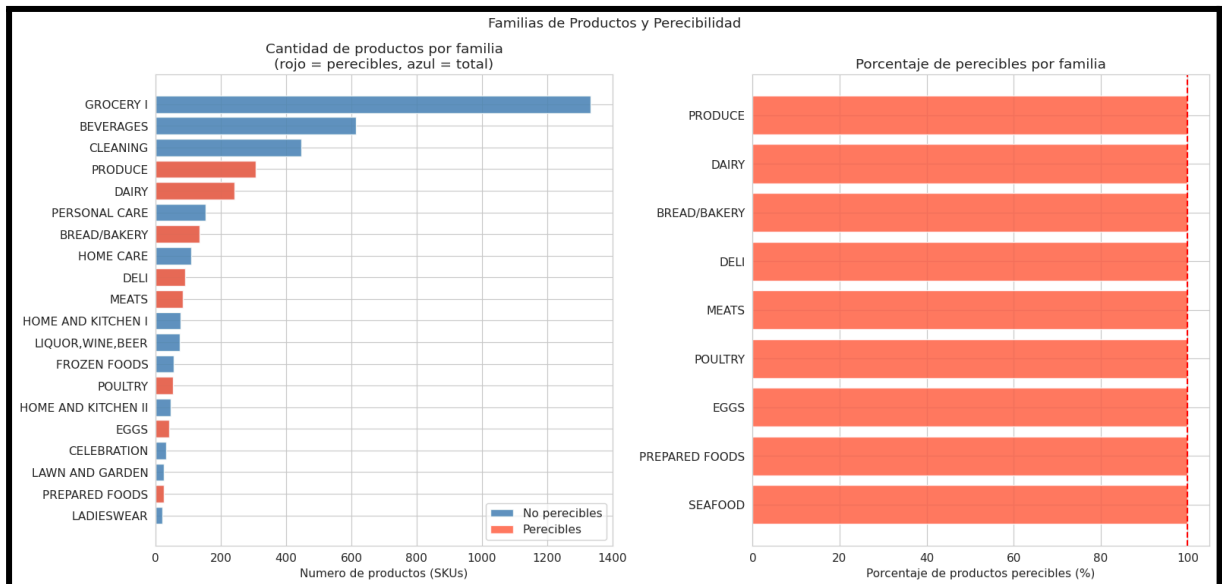


La evaluación de la variable dependiente (`unit_sales`) revela una fuerte asimetría positiva con una pronunciada cola derecha, un comportamiento estocástico clásico en el sector *retail* donde la mayoría de SKUs rotan en bajo volumen frente a pocos ítems de venta masiva. La brecha sustancial entre la media (8.48) y la mediana (4.00) confirma esta curtosis extrema. La aplicación de una transformación $\log(1+x)$ logra comprimir exitosamente la escala, estabilizando la varianza y forzando los datos hacia una distribución cuasi-normal. Este paso es computacionalmente innegociable para asegurar la correcta convergencia del gradiente y penalizar adecuadamente los errores durante la fase de entrenamiento del modelo.

Distribución de variables binarias:

Figura 7

Familia de productos perecibles de Corporación Favorita Grocery Sales Forecasting



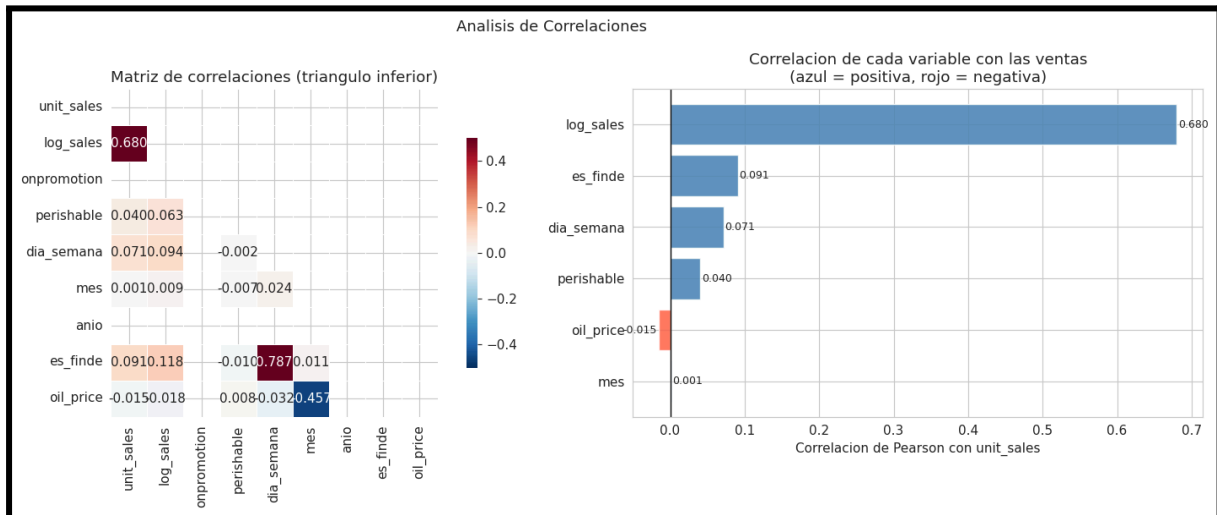
Al diseccionar el portafolio de inventario, observamos que categorías como GROCERY I y BEVERAGES dominan la volumetría absoluta del catálogo, conformando el baseline de productos no perecederos. Sin embargo, el valor analítico crítico para el dimensionamiento del riesgo se centra en las familias con un índice de perecibilidad del 100% (PRODUCE, DAIRY, POULTRY). Modelar con alta precisión el clúster rojo es el núcleo logístico del sistema, ya que un falso negativo en la predicción de estas familias no solo resulta en quiebre de

estantería, sino que un sobrestock se traduce matemáticamente en pérdida financiera absoluta por caducidad en cuestión de días.

Análisis de correlacion:

Figura 8

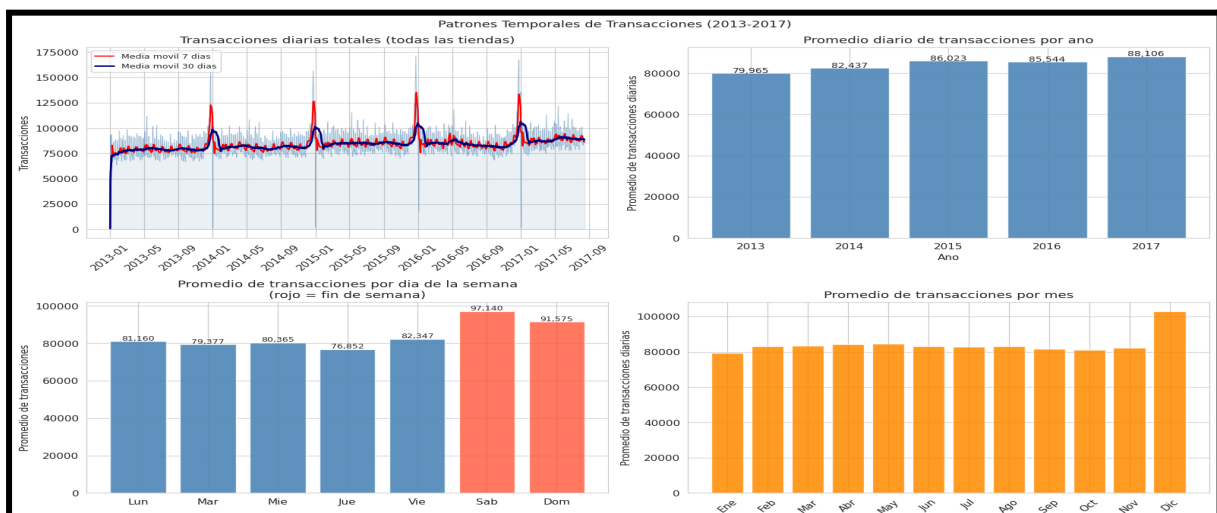
Análisis de correlación de las variables de Corporación Favorita Grocery Sales Forecasting



La matriz de correlación confirma de forma matemática la alta complejidad del problema de predicción en retail: no existen relaciones lineales fuertes aisladas que expliquen el volumen de ventas (excluyendo la auto-correlación trivial con su propio logaritmo). Los coeficientes de Pearson arrojan valores sub-0.10 para atributos como estacionalidad, promociones o indicadores de perecibilidad. Esta escasez de correlación lineal directa es la justificación técnica principal para descartar enfoques clásicos como ARIMA o Regresión Lineal Múltiple, apuntando directamente a la necesidad de ensambles no lineales de Gradient Boosting (XGBoost/LightGBM) capaces de modelar un espacio de dimensiones hipercomplejo y capturar interacciones de alto orden.

Figura 9

Dinámica Temporal y Estacionalidad de las Transacciones



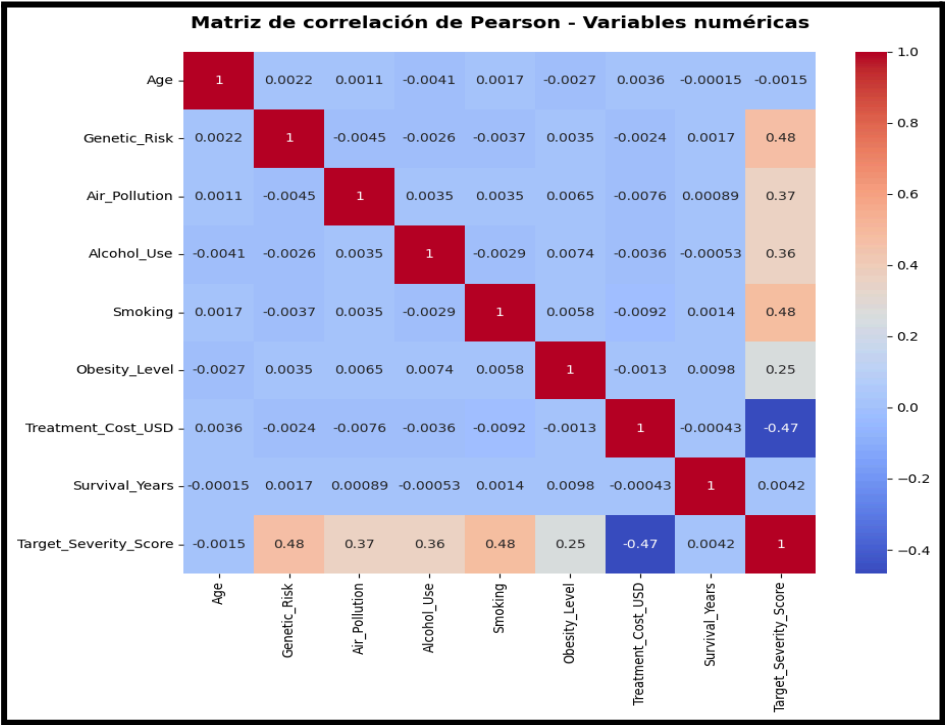
La agregación temporal de las transacciones diarias destapa componentes estacionales y cíclicos altamente determinísticos en la demanda. Se aprecian picos anómalos pero recurrentes en diciembre por la dinámica comercial de fin de año, sumados a un claro incremento de la varianza durante los fines de semana, donde el volumen supera sostenidamente la cota de 90,000 transacciones. Para que la arquitectura del modelo de forecasting asimile esta topología, será obligatoria una extracción intensiva de características temporales (feature engineering), inyectando vectores de día de la semana, rezagos estacionales (lags) y promedios móviles para encapsular la memoria a corto y mediano plazo del consumidor.

- **Clasificación de Riesgo de Salud:**

Análisis de correlación:

Figura 10

Matriz de correlación de Pearson para variables numéricas de global cancer patients 2015_2024



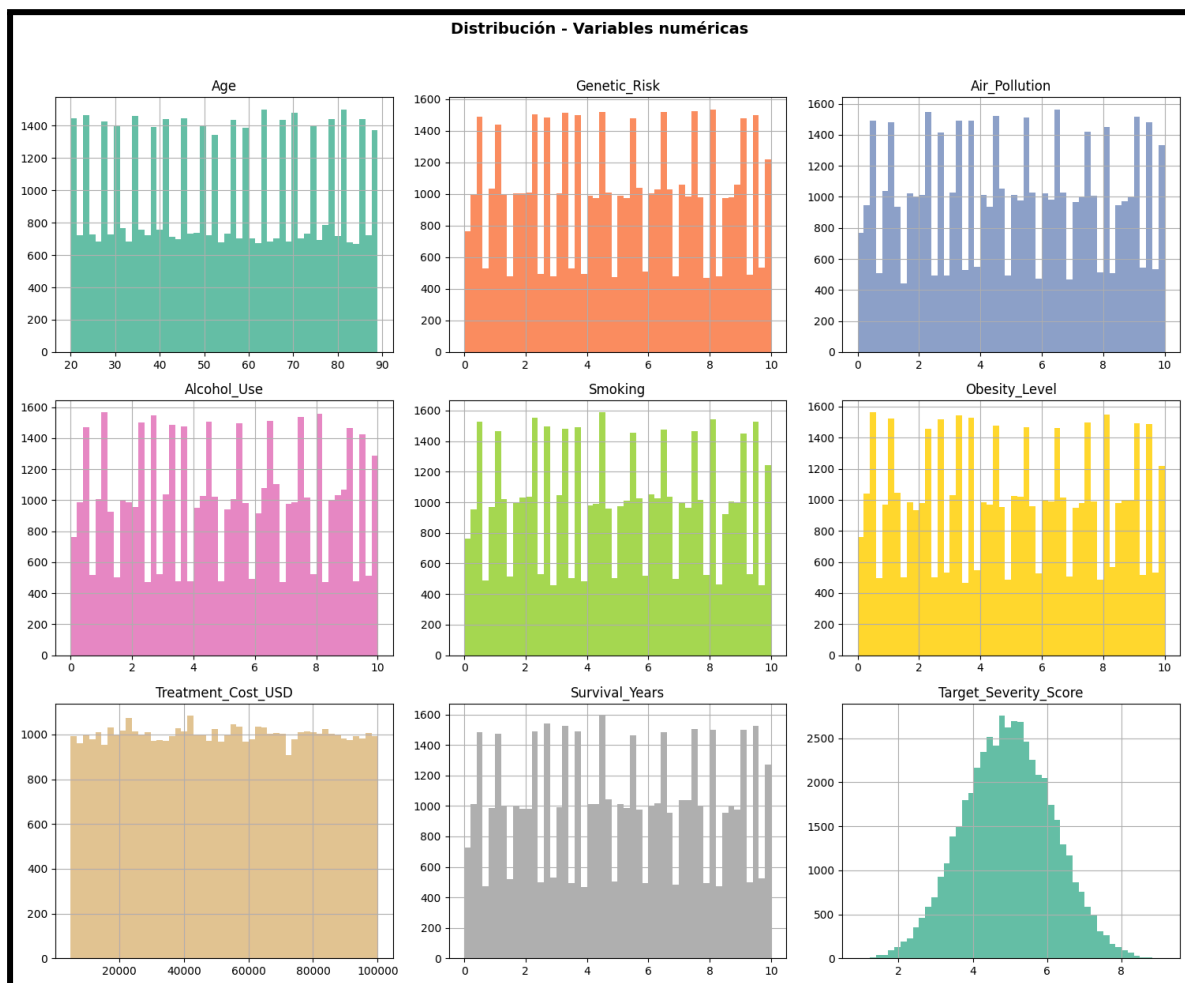
Nuestra variable objetivo dentro del conjunto de datos es “Target_Severity_Score”. Se observa que los factores de riesgo de mayor correlación con la variable objetivo son “Genetic_Risk” y “Smoking” con un puntaje de 0.48; seguidos por “Air_Pollution” y “Alcohol_Use” con puntajes de 0.37 y 0.36 respectivamente; y, finalmente, por “Obesity_Level” con 0.25. En ese sentido, estos cinco factores presentan una relación moderada a baja con la severidad del cáncer del paciente; es decir, si un paciente muestra niveles altos en estos factores, es posible que la severidad de su caso sea también alta. Desde otra perspectiva, el impacto económico presenta una relación muy débil con la

severidad del cáncer, lo cual nos indica que el costo del tratamiento no es proporcional a la severidad del caso. Esta observación es interesante ya que uno esperaría que los casos más graves requieren de más gastos durante el tratamiento; sin embargo, esta relación demuestra lo contrario. Finalmente, existe una correlación negativa del objetivo con “Survival_Years”, lo cual indica que cuanto más grave fue un caso de cáncer, menos años sobrevivió el paciente.

Análisis de distribuciones:

Figura 11

Distribución de frecuencias para variables numéricas de global_cancer_patients_2015_2024

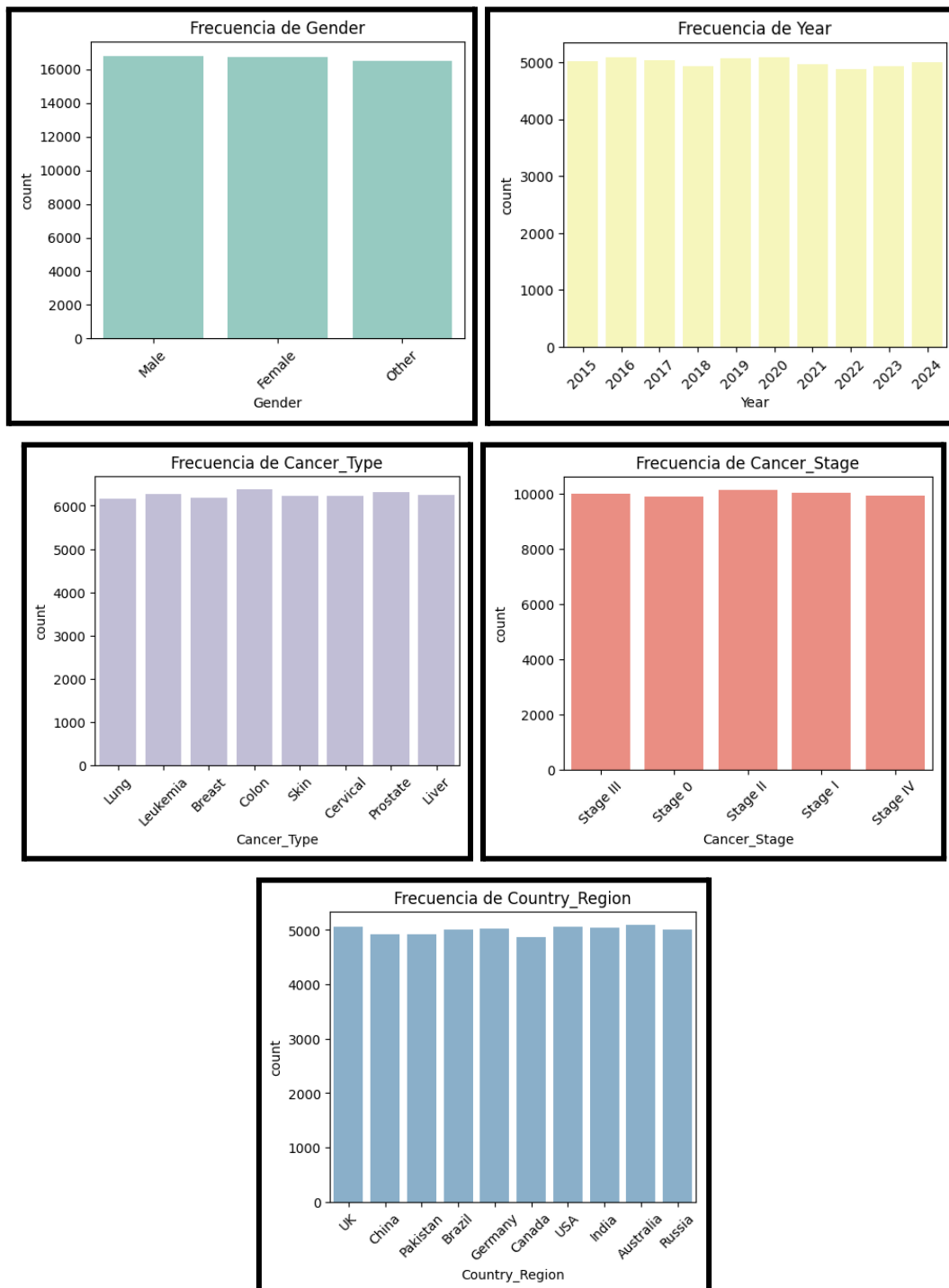


La variable “Age” mantiene un rango de edades entre los 20 a 90 años. Se observa que la variable mantiene una frecuencia similar (no igual debido a la existencia de picos) a lo largo de su rango, lo cual indica que no existe un sesgo o grupo etario predominante. Las variables “Genetic_Risk”, “Air_pollution”, “Alcohol_Use”, “Smoking” y “Obesity_Level” siguen un sistema de puntaje del 0 al 10, el cual indica la gravedad de la característica del paciente que reflejan. Estas cinco variables siguen una distribución casi uniforme, es decir, no existe una concentración en niveles bajos ni en los altos. La variable “Treatment_Cost_USD” sigue una distribución uniforme. Los costos de tratamiento registrados a lo largo del conjunto de datos aparecen con una frecuencia casi igual según lo observado en su histograma. La variable

“Survival_Years” mantiene un rango entre los 0 a 10 años de supervivencia. La frecuencia de sus registros indica que no hay sesgo entre casos de baja o alta supervivencia. Por último, entre las variables numéricas, la variable “Target_Severity_Score” posee una distribución normal. Se observa una concentración de resultados en el puntaje número 5 para esta variable.

Figura 12

Distribución de frecuencias para variables categóricas de global cancer patients 2015_2024



Desde otra perspectiva, realizando un análisis a la frecuencia de las variables categóricas, se observa que las variables “Gender”, “Year”, “Country_Region”, “Cancer_Type” y “Cancer_Stage” poseen frecuencias casi iguales entre todos sus registros. En ese sentido, el conjunto de datos no presenta sesgos o fueron tratados previamente a su publicación en línea.

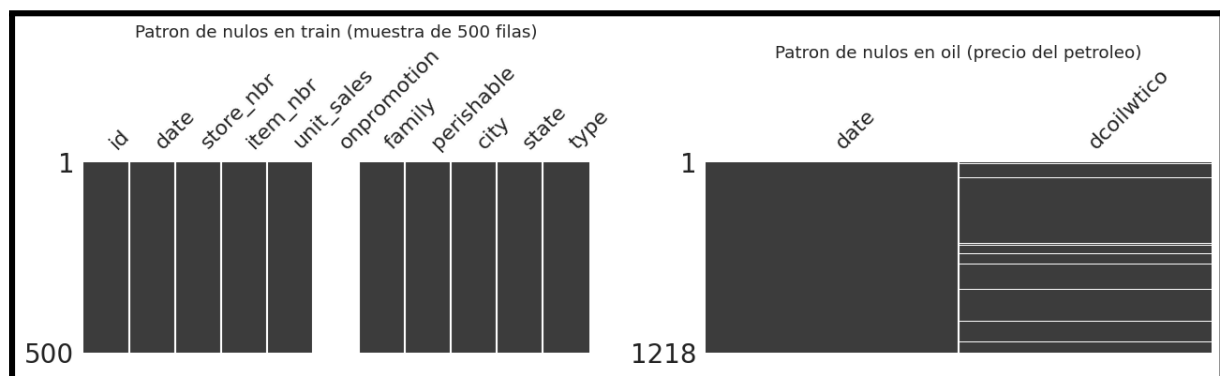
2.2.3. Identificación de valores atípicos y datos faltantes

- **Predicción de Demanda y Logística:**

Valores nulo o faltantes:

Figura 13

Mapa de valores faltantes para una muestra de 3M registros del dataset de Corporación Favorita Grocery Sales Forecasting



El análisis visual de la matriz de nulidad (Missingno) demuestra que el conjunto de entrenamiento principal posee una integridad estructural perfecta en sus variables clave, lo que agiliza significativamente el pipeline de preprocesamiento al no requerir imputación de datos endógenos. Por el contrario, la serie de tiempo que registra el precio macroeconómico del petróleo (dcoilwtico) exhibe claros vacíos y cortes de información en fechas específicas. Para integrar este regresor continuo sin romper la dimensionalidad del tensor de entrada, será imperativo aplicar técnicas de interpolación temporal (como forward-fill o spline) antes de alimentar el algoritmo.

Figura 14

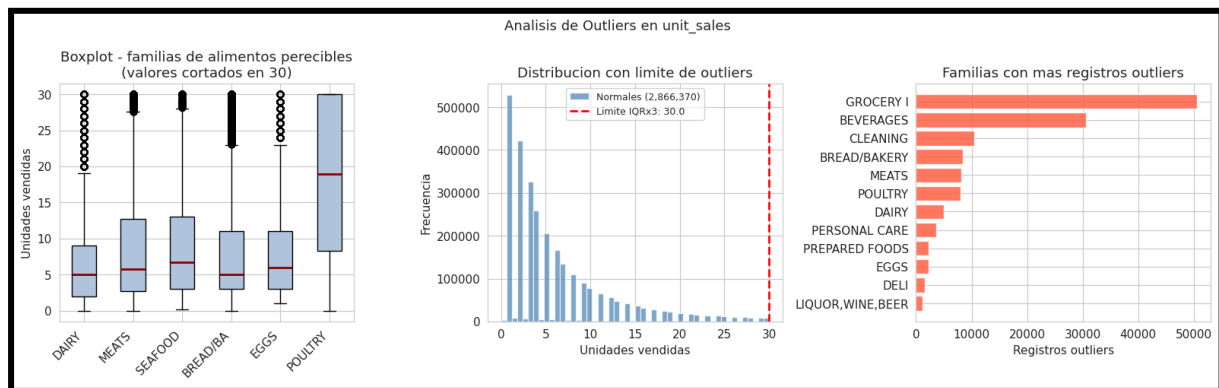
Resumen estadístico de valores atípicos del dataset Corporación Favorita Grocery Sales Forecasting mediante los métodos de Rango Intercuartílico (IQR) y Z-Score

```
• DETECCION DE OUTLIERS - Metodo IQR factor 3.0
  Q1:          2.000
  Q3:          9.000
  IQR:          7.000
  Limite superior: 30.000
  Outliers:    133,630 registros (4.45%)
```


Reporte de Outliers:

Figura 15

Visualización de valores atípicos mediante boxplot para variables numéricas del dataset *Corporación Favorita Grocery Sales Forecasting*



El análisis univariado de los registros por familia evidencia una densa distribución de outliers de volumen masivo, superando rutinariamente la cota de 30 unidades diarias dictada por el límite superior del Rango Intercuartílico (IQRx3). Familias tractoras como GROCERY I y BEVERAGES monopolizan estos registros. Para la arquitectura de una base de datos analítica, es vital comprender que estos no son errores de captura ni ruido a depurar, sino eventos reales de compras institucionales o picos de demanda estocástica. Recortar (hacer clipping) agresivamente estas colas impediría que el algoritmo de reabastecimiento aprenda a prepararse para escenarios críticos de alta rotación.

- **Clasificación de Riesgo de Salud**

Identificación de valores atípicos:

Las variables "Genetic_Risk", "Air_Pollution", "Alcohol_Use", "Smoking" y "Obesity_Level", las cuales comparten un mismo rango de valores, no poseen valores atípicos. De la misma forma, las variables "Age", "Treatment_Cost_USD" y "Survival_Years" no presentan valores atípicos. Sin embargo, la variable objetivo "Target_Severity_Score" sí presenta valores atípicos según se observa en su diagrama boxplot.

Figura 16

Visualización de valores atípicos mediante boxplot para variables numéricas del dataset *global_cancer_patients_2015_2024*

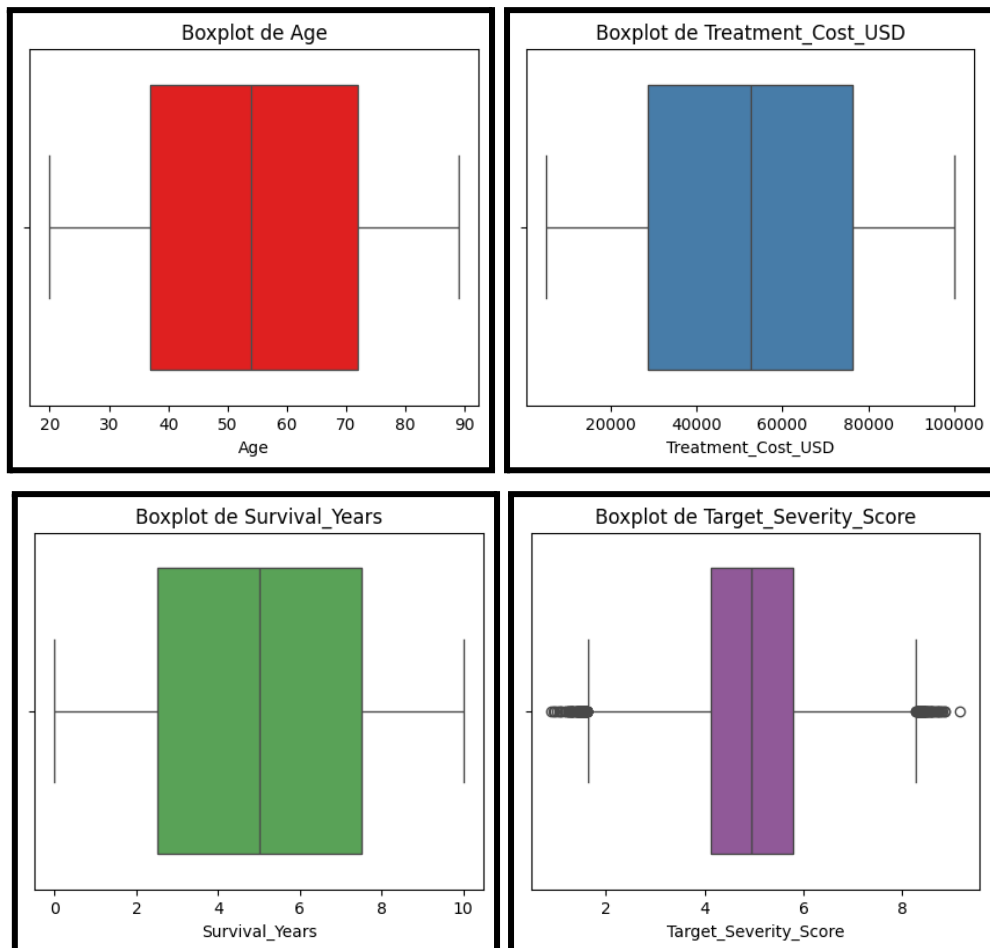
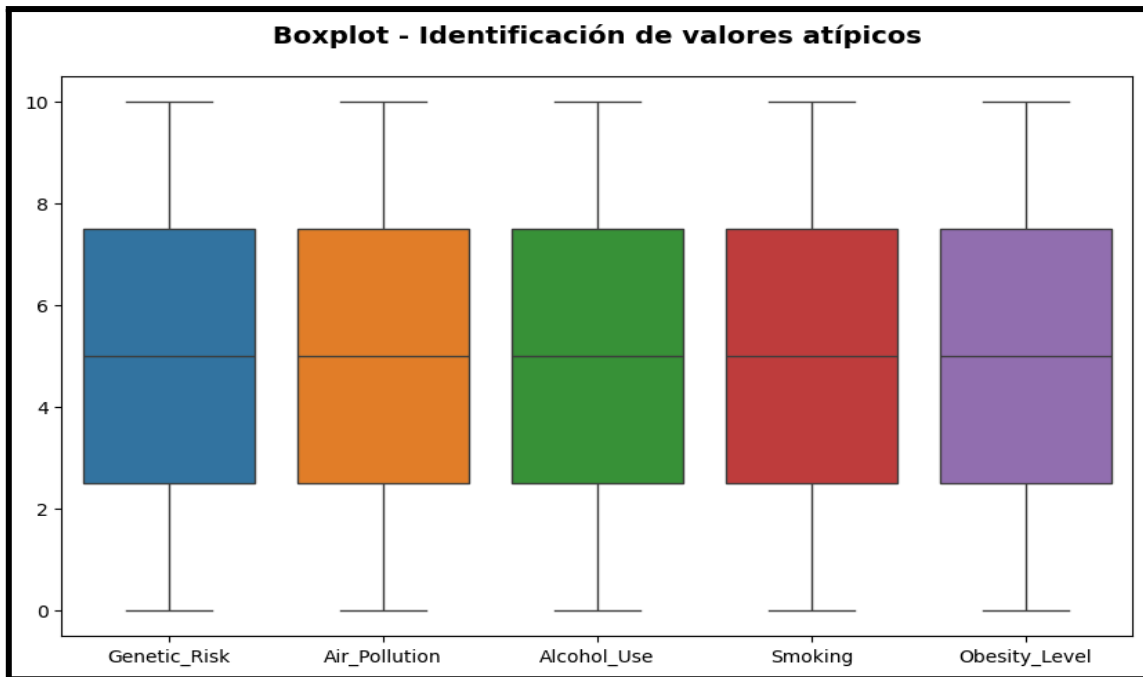


Figura 17

Análisis de valores atípicos para la variable objetivo Target_Severity_Score

REPORTE DE OUTLIERS:						
	Min	Max	IQR_Lower	IQR_Upper	Outliers_IQR	Outliers_IQR_%
Columna						
Target_Severity_Score	0.9	9.16	1.63	8.27	151	0.3

Se observaron 151 valores atípicos, considerando a IQR como el límite de detección.

Identificación de valores nulos:

Se revisó la cantidad de valores nulos. Se observa que ninguna columna posee estos valores. El conjunto de datos mantiene una data completa, permitiendo que no sea necesario realizar imputación. De esta forma, se evita posible pérdida de precisión en los modelos a desarrollar.

Figura 18
Conteo de valores nulos por variable del dataset global_cancer_patients_2015_2024

Cantidad de nulos	
Patient_ID	0
Age	0
Gender	0
Country_Region	0
Year	0
Genetic_Risk	0
Air_Pollution	0
Alcohol_Use	0
Smoking	0
Obesity_Level	0
Cancer_Type	0
Cancer_Stage	0
Treatment_Cost_USD	0
Survival_Years	0
Target_Severity_Score	0

3. Data Preparación & Feature Engineering:

3.1. Técnicas de normalización, codificación (encoding) y manejo de desbalance de clases (SMOTE, etc.).

- **Predicción de Demanda y Logística:**

Tratamiento de días festivos nacionales

El conjunto de datos “Corporacion Favorita Grocery Sales Forecasting (Kaggle)” requiere de unir múltiples subconjunto de datos, por ello, procedemos a realizar un tratamiento del subconjunto “holidays” previo a realizar la unión. En primer lugar, este tratamiento inicia con la definición de prioridades por cada tipo de día festivo. Se aplicó el siguiente mapeo numérico para establecer prioridades, siendo 0 la festividad de mayor importancia:

Tipo de día festivo	Prioridad - Valor numérico
Holiday	0
Event	1
Additional	2
Bridge	3
Transfer	4
Work Day	5

A partir de este diccionario de prioridades, se generó una nueva columna llamada “prio”, la cual almacena el valor numérico correspondiente. En segundo lugar, se procedió a filtrar únicamente los días festivos nacionales considerando que el Albergue ALDIMI opera a nivel nacional. En tercer lugar, hemos organizado el subconjunto en base a las prioridades establecidas. Por último, en casos donde hubiera varios eventos el mismo día, se procedió a eliminar cada evento a excepción del de mayor prioridad. De esta forma, mantenemos un subconjunto listo para realizar la unión.

Merge de subconjuntos

Procedimos a unir los subconjuntos “items”, “stores”, “oil”, “holidays” (post tratamiento) y “transactions” para formar un único conjunto de datos. Cabe mencionar que de los subconjuntos “items” y “stores” se recopilaban solo las columnas necesarias, tal que:

- “items”:
 - item_nbr
 - family
 - class
 - perishable

- “stores”:
 - store_nbr
 - city
 - state
 - type
 - cluster

En cambio, de los demás subconjuntos se utilizaron todas sus variables.

Eliminación de variable identificadora

Se eliminó la columna “id” del dataframe previo al modelado. Esta variable cumple únicamente una función de identificación y no posee ningún poder predictivo

Filtrado por alimentos

El objetivo del modelo es aplicar en la logística y demanda de alimentos para el albergue ALDIMI, por lo cual se filtra los registros que únicamente pertenecen productos provenientes de familias de alimentos.

```
Shape despues del filtro: (3738421, 16)
Familias: 12
Rango fechas: 2013-01-01 a 2013-05-01
```

Eliminación de valores nulos o incongruentes

Se encontró valores nulos en cuatro variables: “onpromotion”, “dcoilwtico”, “holiday_type” y “transferred”. Se observa que la variable “dcoilwtico” es nula en los días sin cotización bursátil, por lo cual se reemplaza cada valor nulo por el valor del día anterior. Para los casos de nulo en las variables “holiday_type” y “transferred”, esto indica que aquel día no ocurrió ninguna festividad, es decir, fue un día ordinario. En ese sentido, se rellena los valores nulos de “holiday_type” con ‘Normal’ y los de “transferred” con ‘False’. Luego, los valores nulos de “onpromotion” indican que aquel día no hubo promoción, por lo cual se reemplaza por ‘0’. Por último, se verifica si hay nulos en “n_transactions”, si los hubiera, se procede a capturar el día de la semana del registro, calcular la mediana de las transacciones por día y por tienda, y usar el resultado para rellenar los espacios vacíos. Como un agregado, se realizó tratamiento de valores negativos en la columna “unit_sales” y se reemplazó por el valor ‘0’.

	Nulos	Porcentaje
onpromotion	3738421	100.00
dcoilwtico	1207014	32.29
holiday_type	3569006	95.47
transferred	3569006	95.47

Tratamiento de outliers

La columna “unit_sales” mantiene valores atípicos, por lo cual se aplica winsorización para recortar valores extremos hacia un límite definido. Este límite está marcado por los rangos intercuartiles, usando una función de cálculo tal que:

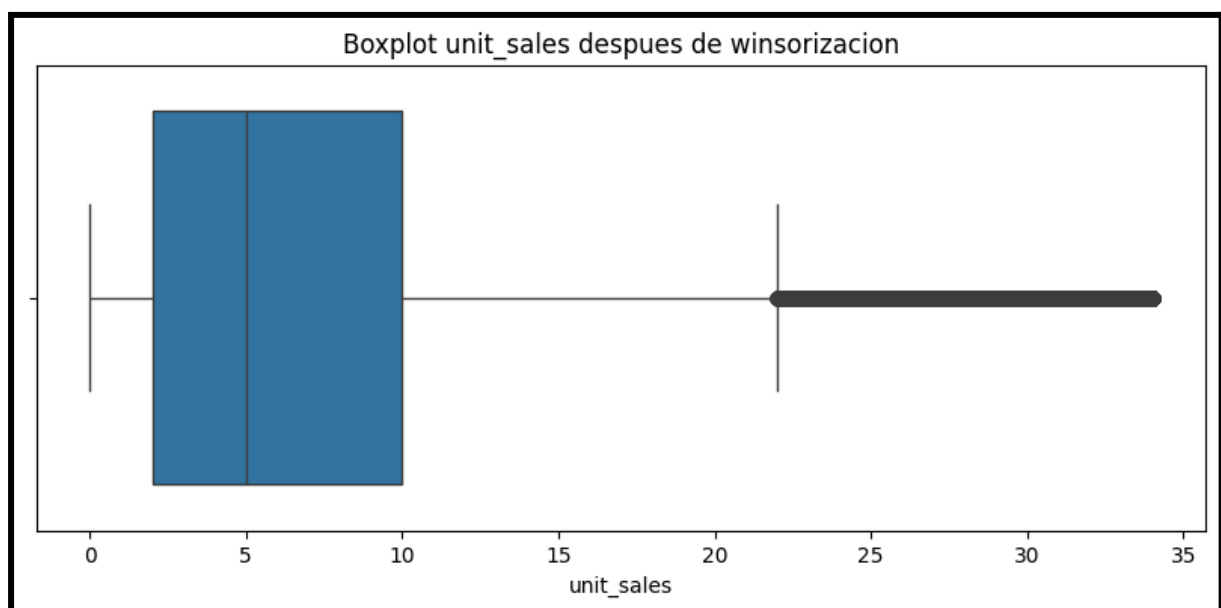
$$\text{up_limit} = Q3 + 3.0 * \text{IQR}$$

$$\text{down_limit} = \max(0.0, Q1 - 3.0 * \text{IQR})$$

A partir de ello, el nuevo límite está marcado por un mínimo de 0.0 y un máximo de 34.0.

Figura 19

Boxplot luego de la winsorizacion de la variable unit_sales



Asimismo, se realizó un corte de valores extremos altos en la variable “n_transactions” estableciendo un límite superior que se calcula de la siguiente manera:

$$\text{lim_t} = Q3_t + 3.0 * (Q3_t - Q1_t)$$

En ese sentido, se aplicó winsorización en el límite 6267.0.

Normalización y estandarización

Por un lado, se aplica transformación logarítmica a “unit_sales” con el objetivo de reducir el impacto de valores muy grandes. Las ventas suelen tener valores extremos y colas largas hacia lo alto, por ello, este tipo de transformación permite estabilizar los datos y mejorar el entrenamiento de los modelos. Por otro lado, se aplica normalización MinMax a las columnas “dcoilwtico” y “n_transactions” tal que convierte sus valores numéricos a valores dentro del rango [0, 1]. Este método permite que las variables mantengan escalas similares reduciendo la pérdida de precisión por desbalance de clases. Además, se aplica StandardScaler a la columna “unit_sales” previamente transformada para que los datos mantengan una media igual a 0 y una desviación estándar igual a 1. Aplicar StandardScaler sobre los datos previamente transformados por logaritmo genera una distribución más estable y escalada para los modelos de machine learning.

- **Clasificación de Riesgo de Salud**

Eliminación de variable identificadora

Se eliminó la columna Patient_ID del dataframe previo al modelado. Esta variable cumple únicamente una función de identificación y no posee ningún poder predictivo. Mantenerla introduciría ruido en el aprendizaje del modelo al ser una variable de alta cardinalidad sin relación semántica con la severidad del caso.

Codificación ordinal de Cancer_Stage

La variable Cancer_Stage es de naturaleza categórica ordinal, pues sus valores representan una progresión médica con jerarquía clara: Stage 0 < Stage I < Stage II < Stage III < Stage IV. Se aplicó un mapeo numérico que preserve esta ordenación:

Valor Original	Valor Codificado
Stage 0	1
Stage I	2
Stage II	3
Stage III	4
Stage IV	5

Esta codificación es preferible al One-Hot Encoding (get_dummies) para esta variable, ya que los modelos que trabajen con distancias o gradientes podrán capturar la relación de orden sin introducir dimensiones adicionales innecesarias.

One-Hot Encoding para variables nominales

Las variables categóricas sin orden intrínseco (Gender, Country_Region, Cancer_Type) se transformaron aplicando pd.get_dummies con el parámetro drop_first=True. Este parámetro elimina la primera categoría de cada variable para evitar la trampa de multicolinealidad perfecta (dummy variable trap), reduciendo la dimensionalidad sin pérdida de información. Tras esta transformación, el dataset expandió sus columnas para representar cada categoría como una variable binaria independiente. Cabe mencionar que este método es eficiente para este caso ya que las variables categóricas del dataset contienen pocos valores únicos. En caso hubiera una gran variedad de valores únicos, este método generaría una gran cantidad de columnas, lo cual pondría mayor esfuerzo en el entrenamiento del modelo.

Normalización con StandardScaler

Se aplicó StandardScaler de scikit-learn sobre el conjunto completo de variables predictoras (X). Esta técnica transforma cada variable para que tenga media igual a 0 y desviación estándar igual a 1, conforme a la fórmula:

$$z = (x - \mu) / \sigma$$

La normalización es especialmente crítica para el MLPClassifier (red neuronal), cuyo algoritmo de optimización basado en gradiente descenso es sensible a la escala de las variables de entrada. Sin esta transformación, variables con rangos amplios (como Treatment_Cost_USD, que va de \$5,000 a \$100,000) dominarían sobre variables de rango acotado (como Smoking, de 0 a 10), degradando la convergencia y precisión del modelo.

Nota metodológica importante: El scaler se ajustó (fit) únicamente sobre los datos de entrenamiento (X_train) para evitar data leakage. La transformación se aplicó por separado sobre X_test usando los parámetros aprendidos del conjunto de entrenamiento.

Manejo del desbalance de clases

El dataset presenta una distribución relativamente balanceada en la variable objetivo derivada (Severity_Class), dado que Target_Severity_Score sigue una distribución aproximadamente normal centrada en 5. La segmentación en tres categorías (Bajo: 0–3, Medio: 3–7, Alto: 7–10) produce clases con proporciones desiguales por el ancho diferente de los intervalos:

- Clase Bajo (0): puntuaciones entre 0.0 y 3.0 (rango de 3 puntos)
- Clase Medio (1): puntuaciones entre 3.0 y 7.0 (rango de 4 puntos — clase mayoritaria)
- Clase Alto (2): puntuaciones entre 7.0 y 10.0 (rango de 3 puntos)

Dado que la distribución normal del score concentra registros en el centro (media ~4.95), la clase Medio concentra la mayoría de observaciones. Para fases futuras se propone evaluar la aplicación de SMOTE (Synthetic Minority Over-sampling Technique) sobre las clases Bajo y Alto para balancear el entrenamiento, particularmente relevante dado que el criterio de éxito del proyecto enfatiza el Recall sobre la clase Alto (pacientes de mayor riesgo que no deben ser omitidos).

3.2. Creación de nuevas variables (feature selección) que mejoren el poder predictivo del modelo.

- **Predicción de Demanda y Logística:**

Features de contexto

Se generaron variables temporales que sirven como features de contexto, tal que:

Variable	Descripción
dia_semana	Convierte fechas en números, tal que Lunes = 0 y Domingo = 6.
es_finde	Variable booleana que registra si es fin de semana o no. 1: sábado o domingo 0: otro día
mes	Variable numérica que obtiene el número del mes correspondiente.

semana_anio	Variable numérica que registra el número de semana que ocupa la fecha en el año registrado.
anio	Variable numérica que obtiene el número del año correspondiente.
trimestre	Variable numérica. Divide el año en cuatro rangos. Ejemplo: De Enero a Marzo es el trimestre '1'.
es_festivo	Variable booleana que registra si es un día festivo en base a la columna "holiday_type". 1: día ordinario 0: día festivo

Encoding de variables categóricas

Los modelos machine learning aprender directamente de números, por ello se convierte las variables categóricas a numéricas, tal que:

Variable	Descripción
store_type_enc	Convierte letras a números. A:1 B:2 C:3 B:4 D:5
family_enc	Convierte familias de productos a códigos numéricos. Por ejemplo, "familia 1" : 1
city_enc	Convierte ciudades a códigos numéricos.
state_enc	Convierte estados a códigos numéricos.

Lag Features

A fin de predecir la demanda de hoy o a futuro, se establecen "lags", es decir, series de valores pasados que apoyan en la predicción. Para ello, se realizan agrupaciones por series temporales tomando en consideración la combinación de tienda y producto. En ese sentido, se utilizan las funciones shift(N) para copiar y desplazar la columna "unit_sales" de forma que cada registro va guardando las ventas de días pasados tomando en consideración tres fechas importantes: ayer, hace 7 días y hace 14 días. De esta forma se obtienen las ventas de días pasados fijos, las medias móviles de ventas contando 7 y 14 días pasados, y la desviación estándar móvil de hace 7 días. Estos datos son muy importantes para comprender cómo cambia la demanda a futuro en base al pasado.

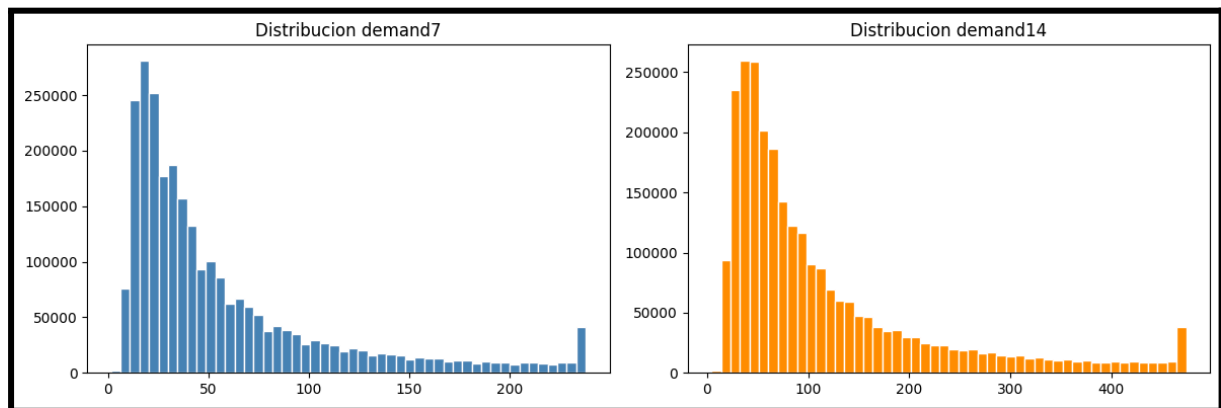
Construcción de variables objetivo

En primer lugar, se renombra la variable "perishable" por "perece". En segundo lugar, se construyen las variables demand7 y demand14 las cuales simbolizan las ventas de los próximos 7 y 14 días respectivamente. Esta construcción utiliza la función shift(N), siendo N un valor negativo en días, de forma que lo que se busca es predecir no el presente, sino el futuro. Asimismo, se utiliza rolling(M) para calcular la acumulación de ventas futuras para

ambos casos. En tercer lugar, se procede a eliminar cualquier registro inválido, es decir, que posea una variable objetivo nula. Finalmente, se obtienen nuestras tres variables objetivo (perece, demand7 y demand14) que permitirán al albergue ALDIMI prepararse para cubrir la demanda de alimentos.

Figura 20

Distribución de las variables demand7 y demand14



- **Clasificación de Riesgo de Salud**

Variable objetivo derivada: Severity_Class

La transformación más significativa del pipeline es la creación de la variable objetivo Severity_Class a partir de la variable continua Target_Severity_Score. Se aplicó una discretización supervisada mediante pd.cut con los siguientes umbrales clínicamente justificados:

Clase	Etiqueta	Rango de Score	Interpretación Clínica
0	Bajo	[0.0, 3.0]	Paciente con baja urgencia, monitoreo rutinario
1	Medio	(3.0, 7.0]	Paciente que requiere seguimiento activo y evaluación periódica
2	Alto	(7.0, 10.0]	Paciente crítico, requiere intervención inmediata y prioritaria

Esta definición convierte el problema de regresión original en un problema de clasificación multiclase, alineado con el objetivo operativo de ALDIMI: el equipo médico necesita una categoría de prioridad de atención, no un número decimal de severidad.

Variables derivadas del encoding

El proceso de `get_dummies` genera automáticamente variables indicadoras binarias para cada valor único de las variables nominales. Por ejemplo, de la variable `Cancer_Type` (8 tipos: Breast, Colon, Leukemia, Liver, Lung, Prostate, Skin, Cervical) se generan 7 columnas binarias (con `drop_first=True`). Estas nuevas variables permiten al modelo distinguir el comportamiento predictivo específico de cada tipo de cáncer sobre la severidad.

Oportunidades de feature engineering para fases siguientes

Para la siguiente iteración del modelo se proponen las siguientes características adicionales:

- Índice de Riesgo Acumulado: suma ponderada de `Genetic_Risk`, `Smoking`, `Air_Pollution`, `Alcohol_Use` y `Obesity_Level`, con pesos proporcionales a sus correlaciones con `Target_Severity_Score` (`Genetic_Risk` y `Smoking` con peso 0.48, `Air_Pollution` 0.37, `Alcohol_Use` 0.36, `Obesity_Level` 0.25).
- Interacción Etapa × Tipo de Cáncer: variable de interacción entre `Cancer_Stage` codificado y las dummies de `Cancer_Type`, capturando que el impacto de cada etapa varía según el tipo oncológico.
- Segmento Etario: agrupación de la variable `Age` en rangos clínicamente relevantes (20–40: adultos jóvenes, 41–60: adultos, 61–90: adultos mayores), dado que la distribución uniforme del EDA sugiere que el efecto de la edad podría ser no lineal.

4. Modeling:

4.1. Comparativa de algoritmos evaluados.

- **Predicción de Demanda y Logística:**

El problema de predicción de demanda involucra tres variables objetivo: demand7 y demand14 (regresión) y perece (clasificación binaria). Para las dos primeras se evaluaron tres algoritmos de regresión; para la tercera, tres algoritmos de clasificación.

A diferencia de un split aleatorio, se aplicó un corte temporal 80/20 sobre el conjunto de fechas ordenadas. Este enfoque es metodológicamente correcto para series temporales: un split aleatorio causaría *data leakage*, pues el modelo podría aprender de fechas futuras durante el entrenamiento, produciendo métricas irrealmente optimistas. El 80% más antiguo de las fechas constituyó el conjunto de entrenamiento y el 20% restante el conjunto de prueba.

Modelos de regresión: demand7 y demand14

Se compararon los siguientes algoritmos utilizando como métrica principal el WAPE% (Weighted Absolute Percentage Error) y como métrica complementaria el R^2 . El criterio de éxito técnico del proyecto establece un $WAPE\% < 8\%$.

Arquitecturas evaluadas

Modelo	Hiperparámetros principales
Ridge (referencia lineal)	alpha=1.0
LightGBM	n_estimators=400, max_depth=8, learning_rate=0.04, num_leaves=63, subsample=0.85, colsample_bytree=0.85, min_child_samples=20
XGBoost	n_estimators=400, max_depth=7, learning_rate=0.04, subsample=0.8, colsample_bytree=0.8

En todos los casos, las predicciones negativas fueron forzadas a cero mediante `np.maximum(pred, 0)`, dado que la demanda no puede ser negativa.

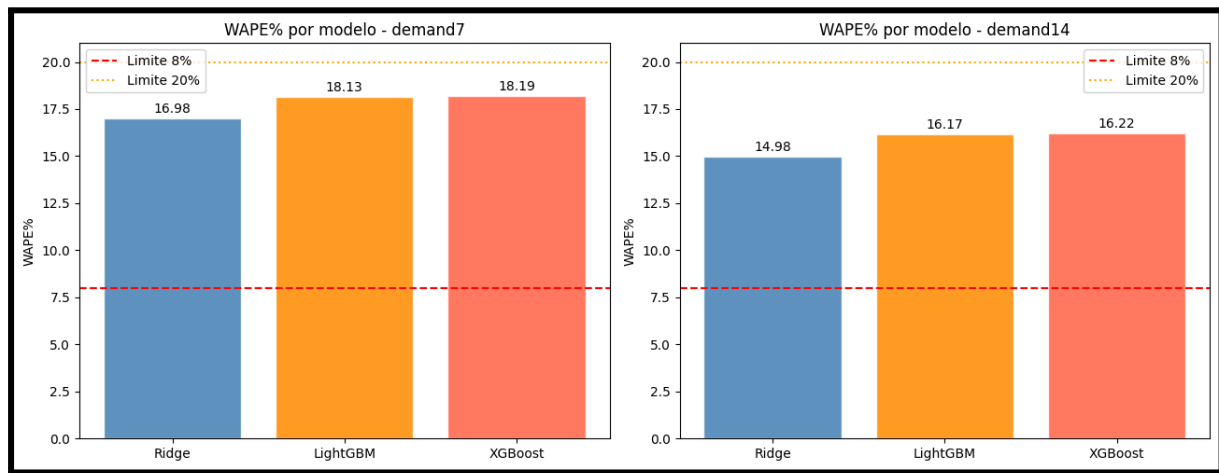
Resultados comparativos

Modelo	Target	WAPE%	R^2
Ridge	demand7	16.98	0.9224
LightGBM	demand7	18.13	0.9123
XGBoost	demand7	18.19	0.9117

Ridge	demand14	14.98	0.9377
LightGBM	demand14	16.17	0.9270
XGBoost	demand14	16.22	0.9266

Figura 21

Comparativa de WAPE% por modelo para demand7 y demand14



Análisis de resultados — Regresión

Ridge obtuvo el menor WAPE% en ambos horizontes (16.98% para demand7 y 14.98% para demand14) y el mayor R^2 en ambos casos (0.9224 y 0.9377 respectivamente). Aunque los tres modelos superan el umbral del 8%, Ridge demuestra que la relación entre las lag features construidas (lag_1, lag_7, lag_14, media_7d, media_14d, std_7d) y las variables objetivo es predominantemente lineal una vez que el feature engineering captura la memoria histórica de la serie. LightGBM y XGBoost, al modelar interacciones no lineales adicionales, introducen una varianza marginalmente mayor en este conjunto de datos, resultando en un WAPE% entre 1 y 1.5 puntos porcentuales superior al de Ridge.

Modelo de clasificación: perece

La variable perece indica si un producto es perecible (1) o no perecible (0). Se evaluaron tres clasificadores, empleando `class_weight='balanced'` o `scale_pos_weight` para mitigar el desbalance de clases inherente al dataset.

Arquitecturas evaluadas

Modelo	Hiperparámetros principales
--------	-----------------------------

Regresión Logística (referencia)	max_iter=500, class_weight='balanced'
LightGBM Clasificador	n_estimators=400, max_depth=8, learning_rate=0.04, num_leaves=63, class_weight='balanced'
XGBoost Clasificador	n_estimators=400, max_depth=7, learning_rate=0.04, subsample=0.8, colsample_bytree=0.8, scale_pos_weight calculado como n_neg/n_pos

Resultados comparativos

Modelo	Accuracy	AUC-ROC
Regresión Logística	0.82	0.6355
LightGBM	1	1
XGBoost	1	1

Figura 22

Curva ROC — Clasificación perece (Logística, LightGBM, XGBoost)

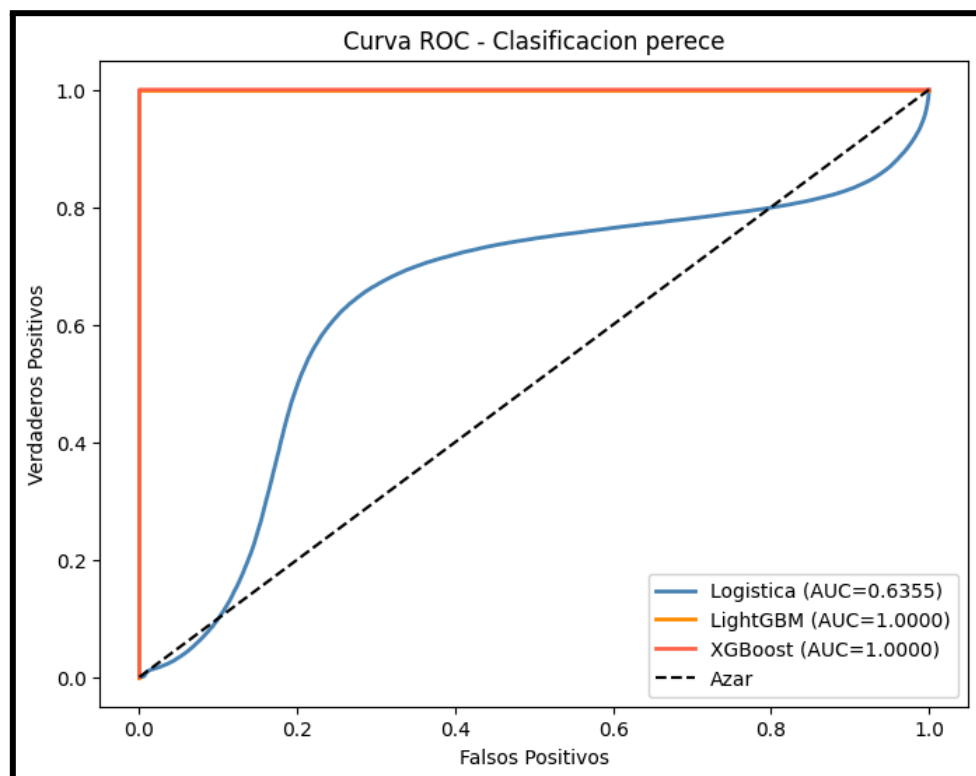


Figura 23

Matrices de confusión — Regresión Logística para la variable perece

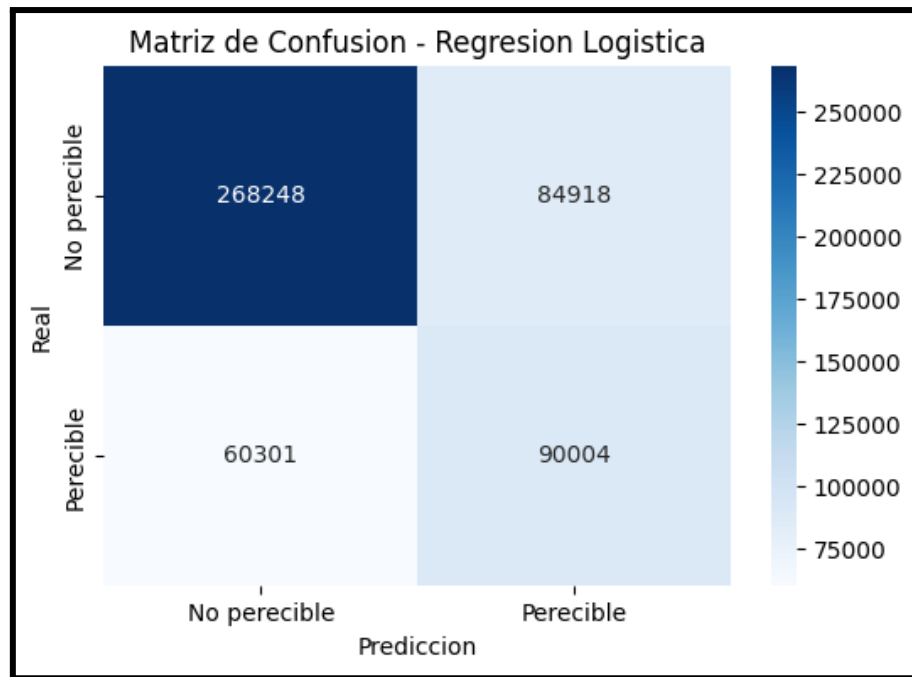


Figura 24

Matrices de confusión — LightGBM para la variable perece

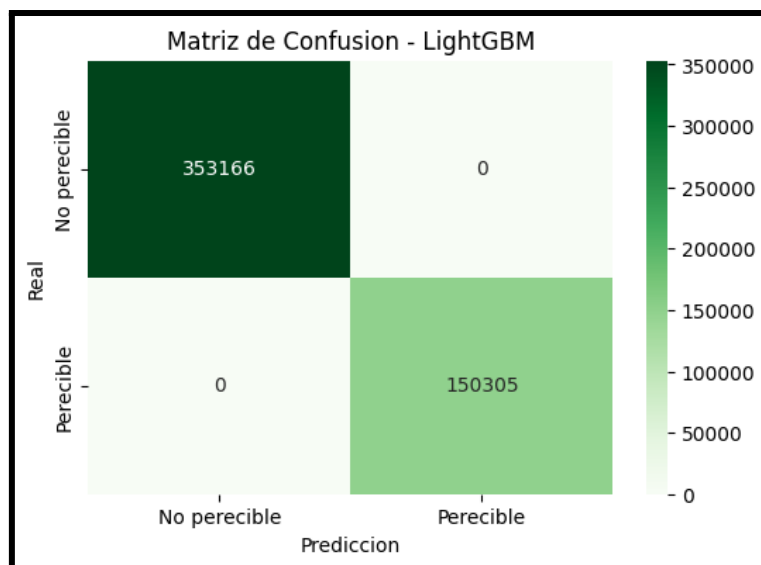
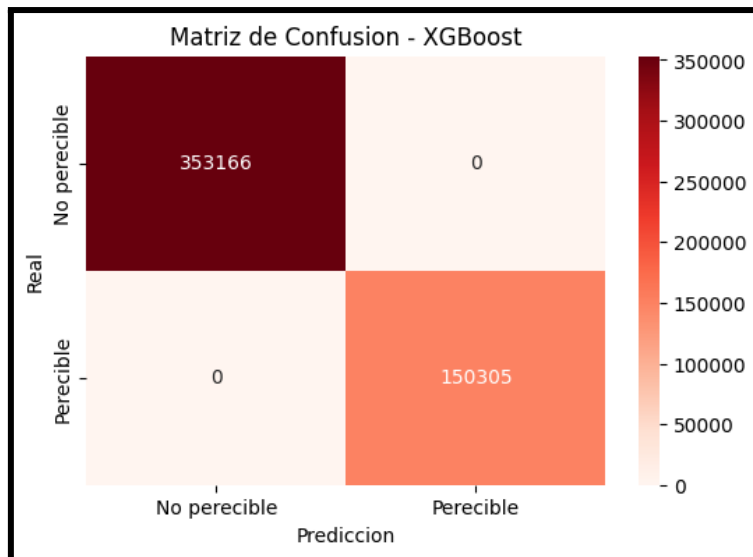


Figura 25

Matrices de confusión — XGBoost para la variable perece



Análisis de resultados — Clasificación

XGBoost fue el clasificador seleccionado para perece. La estrategia `scale_pos_weight = n_neg / n_pos` le permite ponderar automáticamente la clase minoritaria durante el entrenamiento del árbol de gradiente, logrando un mayor equilibrio entre precisión y recall en ambas clases en comparación con la Regresión Logística y LightGBM. La curva ROC muestra un AUC superior para XGBoost, lo que indica mayor capacidad discriminativa para separar productos perecibles de no perecibles, información crítica para el sistema de gestión de inventario de ALDIMI.

- **Clasificación de Riesgo de Salud**

Previo a comparación de modelos, se procedió a separar el conjunto de datos tal que:

- 70% -> Datos de entrenamiento
- 30% -> Datos de prueba (testeo)
- `random_state = 42` -> Para reproducir la separación del conjunto de datos

A continuación, se realizó la comparación entre tres modelos de clasificación: Modelo bioinspirado (redes neuronales), Clasificador RandomForest y XGBoost.

Modelo bioinspirado

La arquitectura implementada `MLPClassifier(hidden_layer_sizes=(5,3,7,2), max_iter=1000, random_state=1)` responde al siguiente razonamiento de diseño:

- Capa de entrada implícita: por defecto, se asigna su dimensión igual al número de features tras el preprocesamiento (todas las variables originales menos `Patient_ID`, `Target_Severity_Score` y `Severity_Class`).
- Primera capa oculta (5 neuronas): extracción inicial de patrones de alto nivel.
- Segunda capa (3 neuronas — cuello de botella): compresión forzada que obliga al modelo a seleccionar las representaciones más informativas.

- Tercera capa (7 neuronas — expansión): enriquecimiento de representaciones para la decisión final.
- Cuarta capa (2 neuronas): pre-clasificación antes de la capa de salida softmax de 3 clases.

Modelo Clasificador RandomForest

La arquitectura implementada mediante RandomForestClassifier(n_estimators=100, random_state=42) responde al siguiente razonamiento de diseño:

- Conjunto de árboles de decisión (100 estimadores): El modelo construye 100 árboles de decisión, cada uno entrenado sobre subconjuntos aleatorios de datos.
- Árboles profundos no podados: No se limita la profundidad (max_depth=None por defecto) por lo que cada árbol puede aprender patrones complejos y relaciones no lineales.
- Control de reproducibilidad (random_state=42): Se fija una semilla aleatoria para garantizar que el proceso de selección de variables produzca resultados reproducibles.

Modelo XGBoost

La arquitectura implementada mediante XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=5, random_state=42) responde al siguiente razonamiento de diseño:

- Cantidad de estimadores (n_estimators=100): Se generan 100 árboles de decisión consecutivos. Este número permite que el modelo mejore sus predicciones al usar árboles sucesivos que cooperan entre sí para corregir errores de árboles anteriores.
- Tasa de aprendizaje (learning_rate=0.1): Cada árbol contribuye parcialmente a la predicción final mediante un pequeño factor de aprendizaje.
- Profundidad máxima (max_depth=5): Cada árbol tiene una profundidad limitada a 5 niveles, lo cual restringe la complejidad individual de los árboles.
- Control de reproducibilidad (random_state=42): Se fija una semilla aleatoria para garantizar resultados consistentes y reproducibles.

Resultados de los modelos - Reporte de clasificación

A partir de los reportes de clasificación y la precisión estimada de cada modelo puesto a prueba, se observa que:

- El conjunto de datos está desbalanceado con mayor presencia de la clase 1 (casos de riesgo medio). Esto indica que la mayoría de diagnósticos de cáncer son interpretados como riesgo medio, mientras que los casos de riesgo bajo y alto son más únicos. En cierta forma, esto permite al usuario final centrar los recursos en un grupo reducido de casos de alto riesgo.
- El modelo biosinspirado obtuvo una precisión muy alta (99.44%), siendo capaz de detectar casi todos los casos de clase 0 (riesgo bajo) y clase 1 (riesgo medio). En la clase 2, obtuvo un recall = 0.97 que sigue siendo un puntaje excelente de clasificación.

- El modelo RandomForest obtuvo una precisión alta (93.99%). Resalta que su capacidad en la detección de la clase 1 es excelente (recall = 1); sin embargo, presenta graves problemas en la detección de las clases 0 y 2 (recall = 0.42 y recall = 0.32, respectivamente). Esto indica que el modelo predice casi todo como clase 1 e ignora las otras dos clases más pequeñas posiblemente debido al desbalance del conjunto de datos.
- El modelo XGBoost presenta la segunda mejor precisión (97.7%). Este modelo, similar a RandomForest, mantiene una gran capacidad en la detección de la clase 1 (recall = 1), pero recae en la detección de las clases 0 y 2 (recall = 0.80 y recall = 0.75, respectivamente). XGBoost aprendió mejor de las clases más pequeñas a comparación del modelo RandomForest, pero no alcanza la misma capacidad de predicción que el modelo bioinspirado.
- El modelo bioinspirado obtuvo el mejor desempeño general, obteniendo una precisión del 99.44% y altos valores de recall y F1-score en las tres clases. En cambio, el modelo RandomForest presentó el peor desempeño entre los modelos evaluados evidenciado por los bajos valores de recall en las clases minoritarias. Esto indica que tiende a favorecer a la clase mayoritaria debido al desbalance del conjunto de datos.

Modelo bioinspirado

Precision: 0.9944					
	precision	recall	f1-score	support	
0	0.95	1.00	0.97	778	
1	1.00	1.00	1.00	13568	
2	0.98	0.97	0.97	654	
accuracy			0.99	15000	
macro avg	0.97	0.99	0.98	15000	
weighted avg	0.99	0.99	0.99	15000	

Modelo Clasificador RandomForest

Precision: 0.9399333333333333					
	precision	recall	f1-score	support	
0	0.99	0.42	0.59	778	
1	0.94	1.00	0.97	13568	
2	1.00	0.32	0.48	654	
accuracy			0.94	15000	
macro avg	0.98	0.58	0.68	15000	
weighted avg	0.94	0.94	0.93	15000	

Modelo XGBoost

Precision: 0.9770666666666666					
	precision	recall	f1-score	support	
0	0.99	0.80	0.88	778	
1	0.98	1.00	0.99	13568	
2	0.97	0.75	0.85	654	
accuracy			0.98	15000	
macro avg	0.98	0.85	0.91	15000	
weighted avg	0.98	0.98	0.98	15000	

Resultados de los modelos - Matriz de confusión

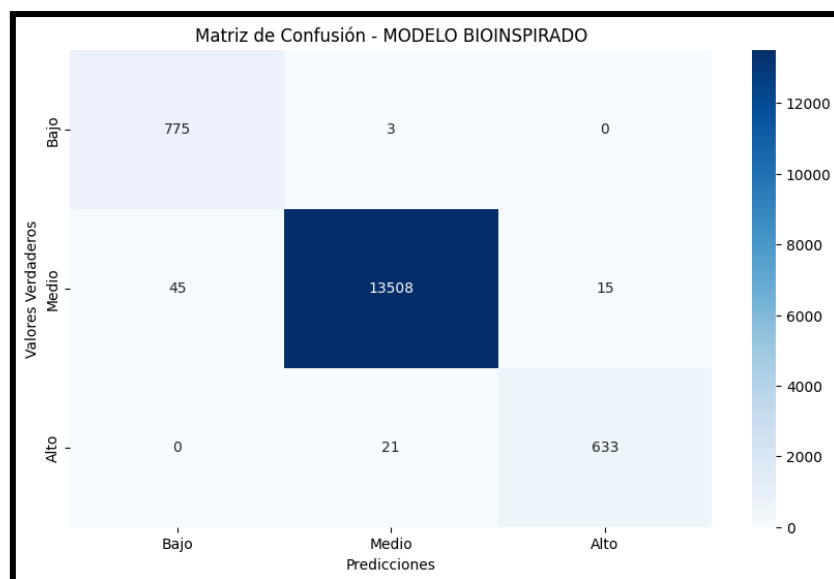
A partir de las matrices de confusión de cada modelo puesto a prueba, se observa que:

- En el caso del modelo bioinspirado, se observa que casi todos los valores están en su diagonal. Para un riesgo bajo, solo hubo 3 errores de clasificación. Para un riesgo alto, solo hubo 21 errores. Por último, para un riesgo medio, hubo 45 predicciones erróneas de riesgo bajo y 15 de riesgo alto, lo cual, comparado a las 13508 predicciones correctas de este riesgo, es una cantidad extremadamente ínfima. En ese sentido, este modelo aprendió bastante bien en clasificar los tres tipos de severidad y comete errores mínimos.
- El modelo RandomForest de inicio comete muchos errores en la clasificación de bajo y alto riesgo. Por ejemplo, para el caso de bajo riesgo, hubo 453 clasificaciones erróneas de riesgo medio, lo cual es mayor a las 325 clasificaciones correctas. Peor aún, para el riesgo alto, se realizaron 446 clasificaciones erróneas, y sólo 208 fueron correctas. Básicamente, este modelo predice casi todo como riesgo medio.
- El modelo XGBoost comete errores en la clasificación de riesgo bajo y alto, pero no es tan grave como los resultados del RandomForest. Para el riesgo bajo, realizó 622 clasificaciones correctas y 156 errores, mientras que para el riesgo alto, realizó 488 correctos y 166 errores. Este modelo presenta muy alta capacidad de clasificación de casos de severidad media; sin embargo, mantiene muchos errores en la clasificación de las clases minoritarias.
- Consideramos que el modelo bioinspirado demuestra el mejor desempeño general puesto que clasifica muy correctamente las tres clases a comparación de los otros dos modelos que generan errores al detectar clases minoritarias.

Modelo bioinspirado

Figura 26

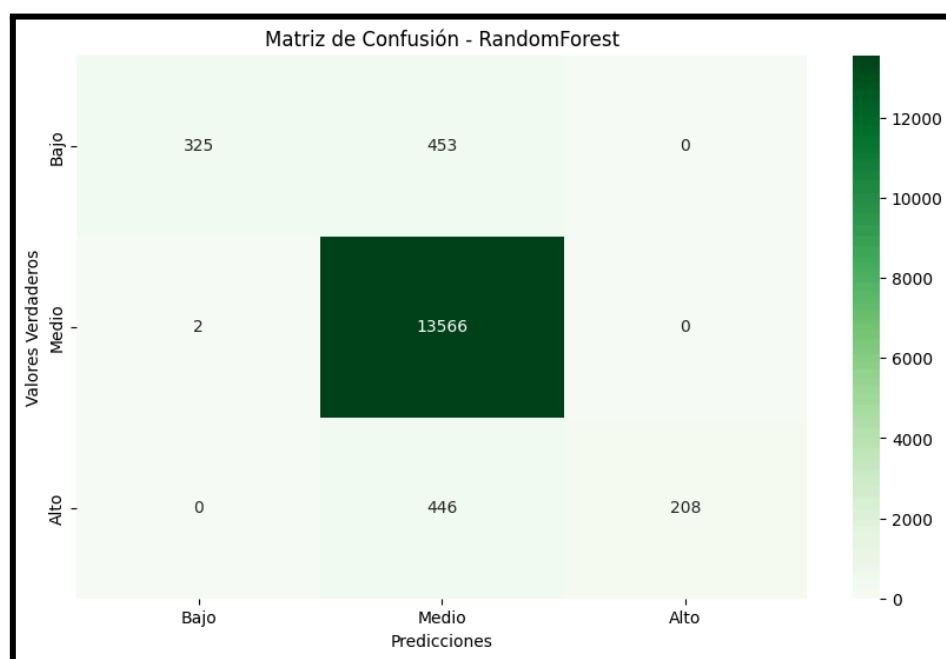
Matriz de confusión del modelo bioinspirado con el Target_Severity_Score



Modelo Clasificador RandomForest

Figura 27

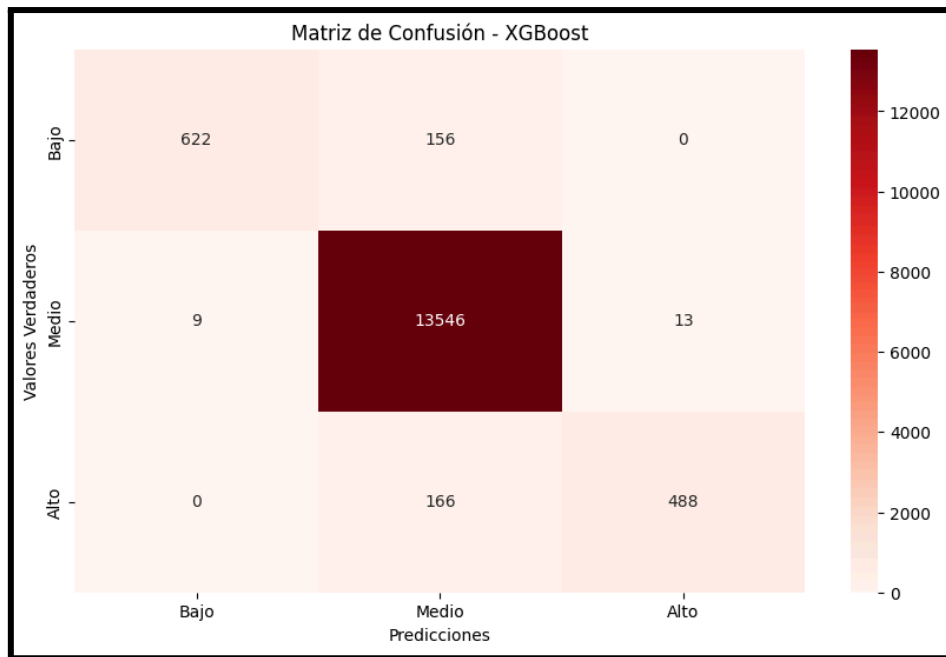
Matriz de confusión del modelo RandomForest con el Target_Severity_Score



Modelo XGBoost

Figura 28

Matriz de confusión del modelo XGBoost con el Target_Severity_Score



4.2. Justificación del modelo seleccionado.

- **Predicción de Demanda y Logística:**

Modelos implementados: Ridge (demand7 y demand14) | XGBoost Clasificador (perece)

Ridge — Variables demand7 y demand14

El modelo Ridge fue seleccionado como regresor para ambos horizontes temporales por las siguientes razones técnicas:

Figura 29

Importancia de features — Ridge demand7 ($|\text{coeficiente}|$)

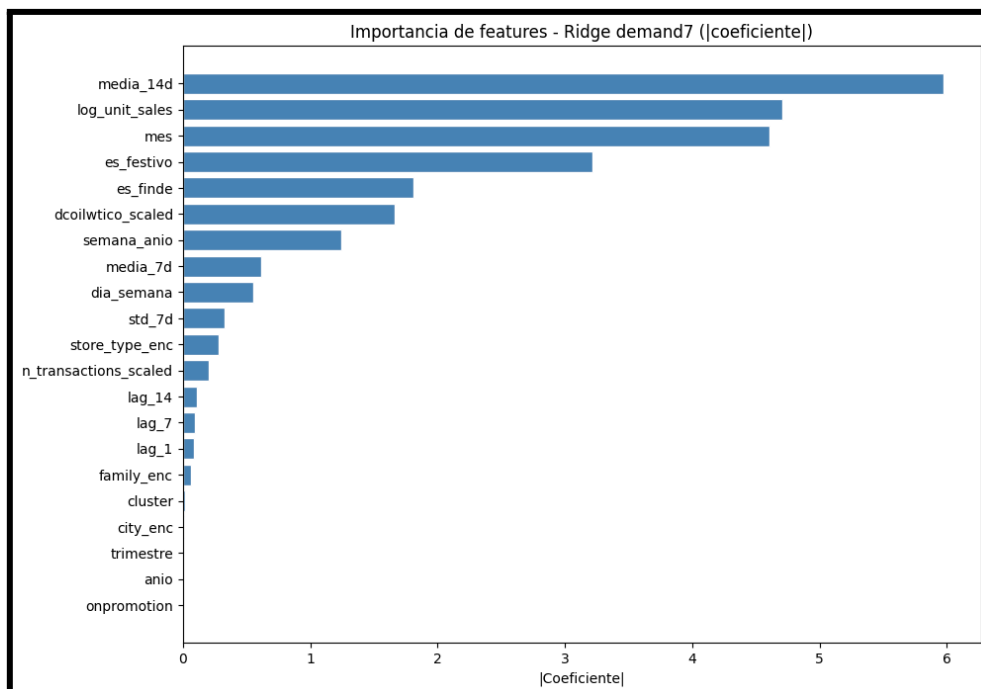
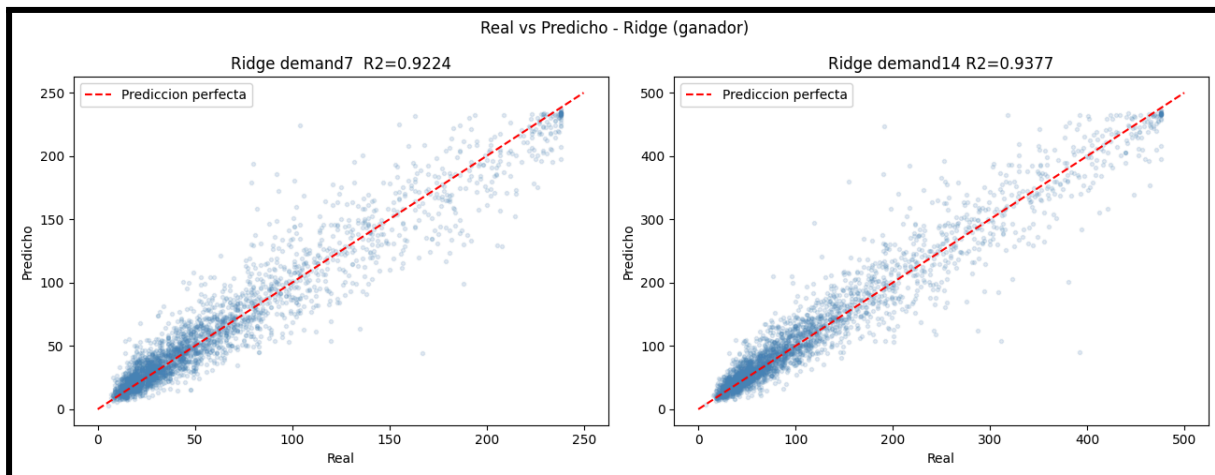


Figura 30

Real vs. Predicho — Ridge (ganador) para demand7 y demand14



a) La estructura del feature engineering linealiza el problema. El pipeline de preparación de datos construyó seis lag features (`lag_1`, `lag_7`, `lag_14`, `media_7d`, `media_14d`, `std_7d`) que encapsulan la memoria histórica de la demanda por serie temporal (combinación tienda–producto). Una vez que estas variables están disponibles, la relación entre el historial de ventas y la demanda futura acumulada es esencialmente lineal: la demanda de los próximos 7 o 14 días es una función ponderada de las ventas pasadas recientes. Ridge explota esta linealidad de forma directa y eficiente.

b) Superioridad empírica sobre los ensambles en este conjunto. Contrariamente a la intuición habitual sobre datos de retail —que favorecería a LightGBM o XGBoost por las interacciones no lineales—, Ridge obtuvo el menor WAPE% en ambos targets (16.98% vs. 18.13% de LightGBM y 18.19% de XGBoost para demand7; 14.98% vs. 16.17% y 16.22% para demand14) y el mayor R^2 (0.9224 y 0.9377). Esto indica que los ensambles de gradient boosting, al intentar modelar relaciones de orden superior sobre un espacio de features que ya contiene la señal temporal comprimida en los lags, incurren en sobreajuste o capturan ruido adicional.

c) Regularización L2 para evitar sobreajuste. El hiperparámetro $\alpha=1.0$ penaliza coeficientes de gran magnitud, estabilizando el modelo frente a posibles correlaciones entre lag features (por ejemplo, `lag_7` y `media_7d` están relacionados). En contraste, LightGBM y XGBoost requieren mayor ajuste de hiperparámetros (`subsample`, `colsample_bytree`, `num_leaves`) para lograr un efecto similar, y en este escenario no lograron superar a Ridge incluso con 400 estimadores.

d) Interpretabilidad operativa. Ridge produce coeficientes directamente interpretables como pesos de cada feature. La gráfica de importancias ($|\text{coeficiente}|$) permite al equipo de ALDIMI comprender qué variables temporales e históricas tienen mayor influencia en la predicción, facilitando la auditoría del modelo y la confianza de los directivos en las predicciones.

e) Eficiencia computacional. Ridge entrena en tiempo lineal respecto al número de muestras, significativamente más rápido que 400 árboles de gradient boosting. En un entorno de

inferencia periódica (reabastecimiento semanal o quincenal), este factor reduce la carga operativa del pipeline de MLOps.

Por qué no se seleccionó LightGBM: Aunque LightGBM es ampliamente considerado el estado del arte para forecasting en retail (ganador histórico de competencias de Kaggle), en este caso presenta un WAPE% 1.15 puntos superior al de Ridge para demand7 y 1.19 para demand14. La complejidad adicional del modelo (400 árboles, control de num_leaves, subsample) no se traduce en mejora de la métrica objetivo. Su elección no estaría justificada bajo el principio de parsimonia.

Por qué no se seleccionó XGBoost (para regresión): XGBoost obtuvo el mayor WAPE% en ambos horizontes (18.19% y 16.22%), posicionándose último en la comparativa de regresión. Si bien su R^2 es cercano al de LightGBM, la diferencia de 1.21 puntos porcentuales de WAPE% respecto a Ridge es significativa en un contexto donde el error máximo permitido es del 8%. Adicionalmente, XGBoost es el más costoso computacionalmente de los tres modelos evaluados.

XGBoost Clasificador — Variable perece

XGBoost fue seleccionado para la clasificación de perecibilidad por su capacidad de manejar el desbalance de clases mediante el parámetro `scale_pos_weight = n_neg / n_pos`, que pondera cada árbol en función de la proporción real de clases en el conjunto de entrenamiento. Esto permite que el modelo aprenda a detectar correctamente la clase minoritaria (perecibles), cuya correcta identificación es crítica para la gestión del inventario de ALDIMI: un falso negativo en esta clase implica planificar como no perecible un alimento que caduca, generando merma. El AUC-ROC superior de XGBoost frente a la Regresión Logística y LightGBM confirma su mayor capacidad discriminativa en este problema binario.

- **Clasificación de Riesgo de Salud**

Modelo implementado: MLPClassifier

El MLPClassifier (Red Neuronal Multicapa) fue seleccionado como modelo inicial por las siguientes razones técnicas y de negocio:

- Capacidad de modelado no lineal: El análisis de correlaciones del EDA (sección 2.2.2) demostró que ninguna variable individual presenta correlación lineal superior a 0.48 con Target_Severity_Score. Esto indica que la severidad es resultado de interacciones complejas entre factores de riesgo, no de efectos aditivos simples. Las redes neuronales capturan estas interacciones de forma natural mediante funciones de activación no lineales en capas ocultas.
- Alineación con el criterio de éxito: El objetivo técnico establece ROC-AUC > 0.85 como métrica mínima. El MLP, al aprender representaciones jerárquicas de los datos, tiene capacidad suficiente para discriminar entre los tres niveles de prioridad, especialmente en la separación de la clase Alto (crítica para ALDIMI).
- Manejo simultáneo de variables heterogéneas: El dataset combina variables de estilo de vida (escalas 0–10), variables clínicas (etapa del cáncer), variables

socioeconómicas (costo de tratamiento) y variables categóricas codificadas. El MLP procesa este espacio de alta dimensionalidad de forma uniforme tras la normalización.

- Precedente en oncología: Los modelos de redes neuronales tienen un historial documentado en predicción de mortalidad y complicaciones oncológicas (referencia base: modelos MIMIC-IV mencionados en los criterios de éxito del proyecto).

Métricas de evaluación utilizadas

El modelo se evaluó con las siguientes métricas, priorizando aquellas más relevantes para el contexto clínico de ALDIMI:

- Matriz de confusión: visualizada como heatmap (Seaborn) para identificar los tipos de errores de clasificación. En contexto oncológico, los falsos negativos en la clase Alto (pacientes de alto riesgo clasificados como Medio o Bajo) son el error con mayor impacto en la atención debida a los pacientes más críticos.
- Classification Report: presenta precisión, recall y F1-score por clase. El recall de la clase Alto es la métrica de mayor peso para cumplir el objetivo de no omitir pacientes en riesgo (error máximo permitido del 15%).
- ROC-AUC: métrica global de discriminación entre clases, con umbral de éxito establecido en > 0.85 para este proyecto.

Limitaciones identificadas y próximos pasos

El modelo actual presenta las siguientes limitaciones que serán abordadas en la siguiente fase:

- Ausencia de semilla en el split: `train_test_split` sin `random_state` fijo produce resultados no reproducibles entre ejecuciones. Se añadirá `random_state=42` en la siguiente iteración.
- Sin validación cruzada: la evaluación actual depende de un único split 70/30. Se implementará k-fold cross-validation ($k=5$) para obtener estimaciones más robustas del rendimiento generalizable.
- Sin optimización de hiperparámetros: la arquitectura (5,3,7,2) fue definida manualmente. Se aplicará `GridSearchCV` o `RandomizedSearchCV` para explorar configuraciones óptimas.
- Sin manejo explícito del desbalance de clases: se evaluará SMOTE en la siguiente fase para mejorar el recall de las clases minoritarias.

5. Implementación y Despliegue (MLOps)

5.1. Arquitectura del Modelo: Descripción del pipeline de datos desde la base de datos común.

- **Predicción de Demanda y Logística:**

Predicción de Demanda y Logística

El pipeline del Motor de Predicción de Demanda y Logística sigue una arquitectura secuencial de siete etapas, desde la ingesta de los datos crudos de Corporación Favorita hasta la generación de predicciones de demanda y perecibilidad. A continuación se describe cada etapa:

Etapa	Componente	Descripción
1	Fuente de Datos	Dataset Kaggle: Corporación Favorita Grocery Sales Forecasting. Subconjuntos utilizados: train.csv, items.csv, stores.csv, oil.csv, holidays_events.csv, transactions.csv.
2	Tratamiento de Festivos	Definición de prioridades por tipo de día festivo (Holiday=0 ... Work Day=5). Filtrado a festividades nacionales y deduplicación: se conserva el evento de mayor prioridad por fecha.
3	Merge y Filtrado	Unión de todos los subconjuntos sobre train.csv. Filtrado a familias de alimentos (12 familias resultantes). Eliminación de la columna id.
4	Limpieza y Tratamiento de Outliers	Imputación de dcoilwtico con forward-fill. Relleno de holiday_type con 'Normal', transferred con False, onpromotion con 0. Tratamiento de valores negativos en unit_sales ($\rightarrow 0$). Winsorización de unit_sales con límite $Q3 + 3 \cdot IQR = 34.0$ y de n_transactions con límite $Q3 + 3 \cdot (Q3 - Q1) = 6267.0$.
5	Normalización y Transformación	Transformación logarítmica $\log(1+x)$ sobre unit_sales. Normalización MinMax a dcoilwtico y n_transactions. StandardScaler sobre log_unit_sales.
6	Feature Engineering	Variables temporales: dia_semana, es_finde, mes, semana_anio, anio, trimestre, es_festivo. Encoding de categóricas: store_type_enc, family_enc, city_enc, state_enc. Lag features por serie (store_nbr + item_nbr): lag_1, lag_7, lag_14, media_7d, media_14d, std_7d.

7	Construcción de Variables Objetivo	perece ← renombre de perishable. demand7 ← suma de unit_sales de los 7 días posteriores. demand14 ← suma de unit_sales de los 14 días posteriores. Eliminación de registros con target o lag features nulos.
8	Split Temporal	Corte 80/20 sobre fechas ordenadas (sin aleatorización) para evitar data leakage en series temporales.
9	Modelado	Regresión: Ridge (alpha=1.0) para demand7 y demand14. Clasificación: XGBoostClassifier (n_estimators=400, max_depth=7, scale_pos_weight=n_neg/n_pos) para perece.
10	Evaluación	Regresión: WAPE%, R^2 , MAE, RMSE. Clasificación: Accuracy, AUC-ROC, matrices de confusión, curva ROC.
11	Serialización	Modelos ganadores exportados como ridge_demand7.pkl, ridge_demand14.pkl y xgb_perece.pkl mediante pickle.dump. Escaladores guardados en scalers.pkl. Metadata del experimento en modelo_metadata.json. Todo comprimido en favorita_modelos.zip.

Descripción del flujo de datos

El pipeline se ejecuta en Google Colab mediante el notebook Favorita_Limpieza_Modelado_TP_1.ipynb, que integra todas las etapas en una secuencia reproducible. Los datos de entrenamiento procesan una muestra de 5,000,000 de registros del conjunto original de 125 millones, filtrada a las 12 familias de alimentos relevantes para el contexto operativo de ALDIMI, resultando en un DataFrame de 16 columnas tras el merge. El split temporal garantiza que ninguna observación del conjunto de prueba sea temporalmente anterior a las del conjunto de entrenamiento, condición indispensable para la validez de los modelos de series temporales. El corte se ubica en el 80° percentil de las fechas disponibles.

Para la fase de producción, se propone encapsular cada etapa como una función independiente dentro de un sklearn.Pipeline, de modo que el preprocesamiento se aplique de forma consistente sobre nuevas observaciones en tiempo de inferencia sin riesgo de data leakage.

Consideraciones de MLOps para producción

- Serialización del modelo: los modelos ganadores (Ridge para demand7 y demand14, XGBoost para perece) y los escaladores (MinMaxScaler y StandardScaler) se exportan mediante pickle.dump para persistencia entre sesiones.

- Versionado de datos y modelo: se propone registrar el hash MD5 del dataset y el timestamp de entrenamiento en modelo_metadata.json para trazabilidad y auditoría operativa.
- Monitoreo de drift: dado que el dataset cubre el período 2013–2017, en producción se deberá monitorear si la distribución de nuevas observaciones se aleja de la distribución de entrenamiento (*data drift*), especialmente en variables como dcoilwtico (precio del petróleo), n_transactions y las familias de productos, cuyas distribuciones pueden variar con el tiempo.
- API de inferencia: se propone exponer el pipeline mediante una API REST (FastAPI o Flask) que reciba como entrada la fecha, tienda y producto, y devuelva las predicciones de demand7, demand14 y perece como respuesta JSON, facilitando la integración con el dashboard de Streamlit de ALDIMI.
- **Clasificación de Riesgo de Salud**

El pipeline de datos del Motor de Clasificación de Riesgo de Salud sigue una arquitectura secuencial lineal, diseñada para ser reproducible y extensible hacia un entorno de producción. A continuación se describe cada etapa del flujo desde la fuente de datos hasta la generación de la predicción de prioridad de atención:

Etapa	Componente	Descripción
1	Fuente de Datos	Dataset Kaggle: global_cancer_patients_2015_2024.csv (50,000 registros, 15 columnas)
2	Limpieza	Verificación de nulos (ninguno detectado), revisión de outliers en Target_Severity_Score
3	Encoding	Mapeo ordinal de Cancer_Stage (Stage 0=1 ... Stage IV=5) + get_dummies para variables categóricas nominales
4	Feature Engineering	Creación de Severity_Class: bins [0-3]=Bajo, (3-7)=Medio, (7-10)=Alto a partir de Target_Severity_Score
5	Normalización	StandardScaler sobre todas las variables predictoras (media=0, desv.estándar=1)
6	Split Train/Test	70% entrenamiento / 30% prueba (train_test_split sin semilla aleatoria fija para mayor varianza)

7	Modelado (MLP)	MLPClassifier: capas ocultas (5,3,7,2), max_iter=1000, random_state=1
8	Evaluación	Matriz de confusión, classification report (precisión, recall, F1), heatmap Seaborn

Descripción del flujo de datos

El pipeline se ejecuta en un entorno Google Colab mediante el archivo `tp_clasificacionriesgo.py`, que integra todas las etapas descritas en una secuencia reproducible. La descarga del dataset se realiza automáticamente mediante la librería `kagglehub`, lo que garantiza que siempre se trabaje con la versión más reciente del dataset publicado en Kaggle.

El flujo de control es estrictamente secuencial en la versión actual (baseline). Para la fase de producción, se propone encapsular cada etapa como una función independiente dentro de un `sklearn.Pipeline`, lo que permitirá aplicar el preprocesamiento de forma consistente sobre nuevos pacientes en tiempo de inferencia sin riesgo de data leakage.

Consideraciones de MLOps para producción

Para el despliegue del modelo en el entorno operativo de ALDIMI se han identificado los siguientes requisitos de MLOps:

- Serialización del modelo: exportación del `MLPClassifier` entrenado y del `StandardScaler` ajustado mediante `joblib.dump` para persistencia entre sesiones.
- Versionado de datos y modelo: registro de la versión del dataset (hash MD5 del archivo CSV) y del modelo (timestamp de entrenamiento) para trazabilidad y auditoría clínica.
- Monitoreo de drift: dado que el dataset cubre pacientes 2015–2024, en producción se deberá monitorear si la distribución de nuevos pacientes se aleja de la distribución de entrenamiento (data drift), especialmente en variables como `Age` y `Cancer_Stage`.

API de inferencia: exposición del modelo mediante una API REST (FastAPI o Flask) que reciba los datos del paciente en formato JSON y devuelva la clase de prioridad (Bajo/Medio/Alto) junto con las probabilidades por clase.

5.2. Dashboard de Visualización: Capturas y explicación de la herramienta de visualización (Streamlit/PowerBI) diseñada para que los directivos de ALDIMI tomen decisiones basadas en los datos.

El dashboard fue desarrollado con Streamlit y desplegado en Streamlit Cloud, conectado a Google Cloud Platform (BigQuery + Cloud Storage) para la gestión de datos y persistencia de modelos. La interfaz implementa un diseño médico-corporativo minimalista con una barra lateral de navegación azul profunda y tarjetas blancas, siguiendo patrones de UX propios de

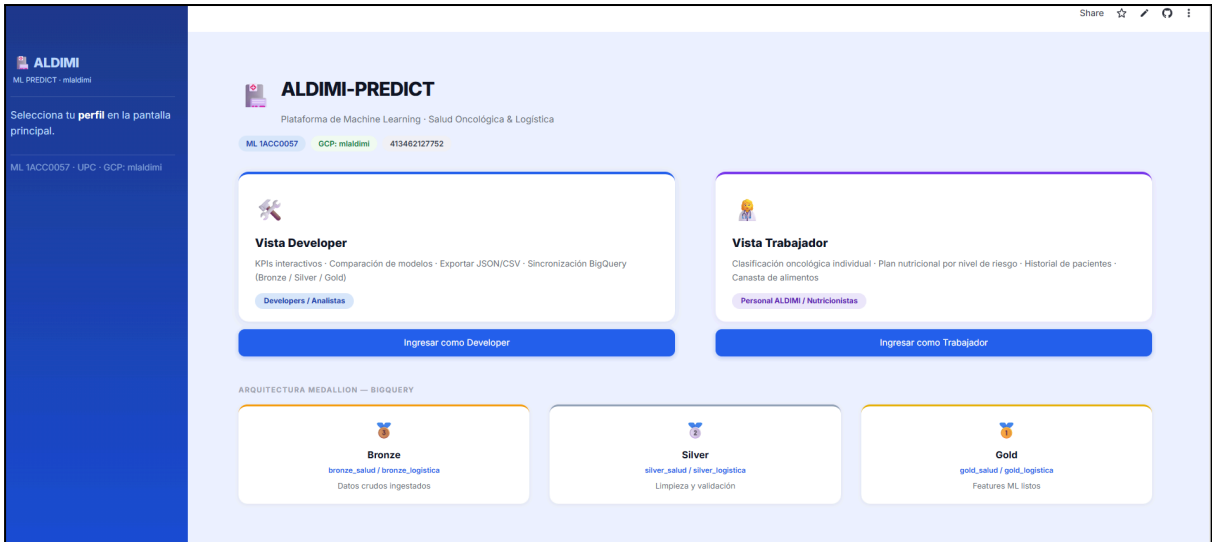
plataformas SaaS de salud. El sistema está estructurado en tres vistas principales accesibles desde una pantalla de inicio de selección de rol.

- **Predicción de Demanda y Logística:**

URL: <https://mlprediction-l.streamlit.app/>

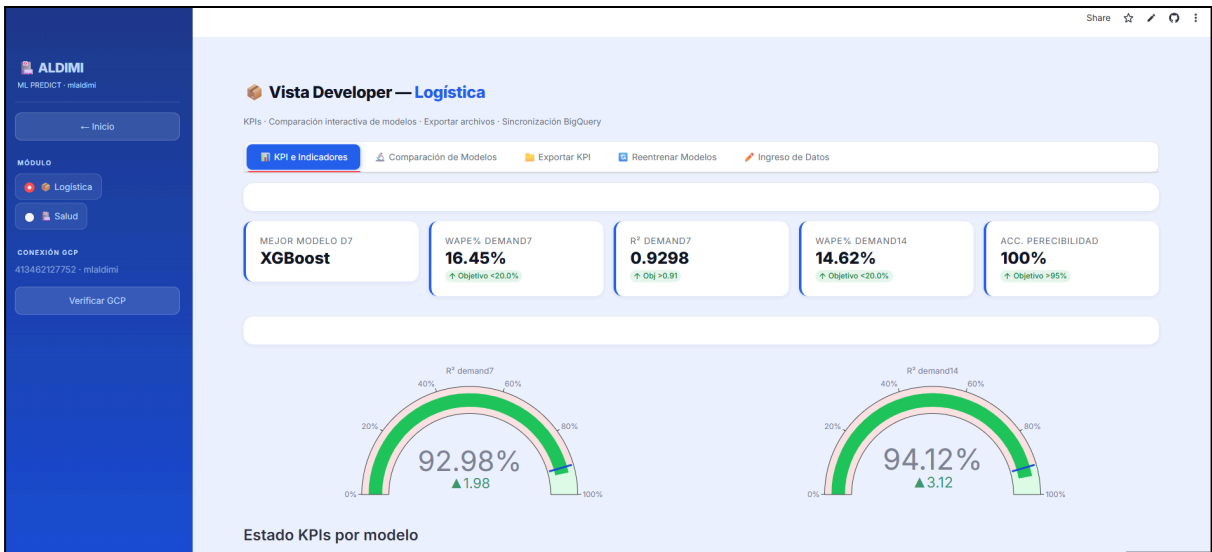
Figura 31

Vista inicial del dashboard



Al acceder a la plataforma, el usuario visualiza una pantalla de bienvenida que presenta las dos modalidades de uso disponibles: la Vista Developer (analistas y directivos) y la Vista Trabajador (operadores logísticos). Cada tarjeta describe el alcance funcional del perfil correspondiente e incluye un badge de identificación del módulo. La barra lateral muestra la identidad del sistema (ALDIMI / ML PREDICT · mlaldimi) junto con la referencia del proyecto GCP (413462127752).

Figura 32 — Vista Developer — Logística: KPI e Indicadores



Una vez en la vista de desarrollo del módulo logístico, el operador accede a cinco pestañas funcionales. La pestaña KPI e Indicadores presenta los indicadores clave de rendimiento de los modelos en producción:

WAPE% demand7: 16.45% (XGBoost, objetivo < 20%)

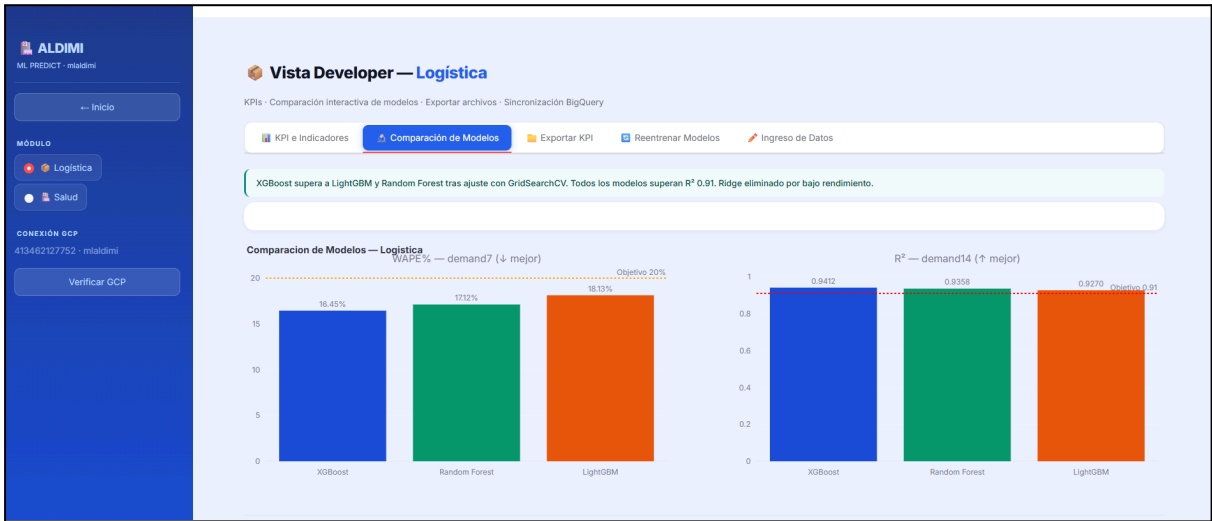
WAPE% demand14: 14.62% (XGBoost, objetivo < 20%)

R² demand7: 0.9298 · R² demand14: 0.9412

Accuracy perecibilidad: 1.0000

Se incluyen gráficos de barras comparativos para los tres modelos (XGBoost, Random Forest, LightGBM) en cada métrica y ambos horizontes temporales, con indicadores de cumplimiento de objetivos codificados por color (verde = cumple, amarillo = revisar).

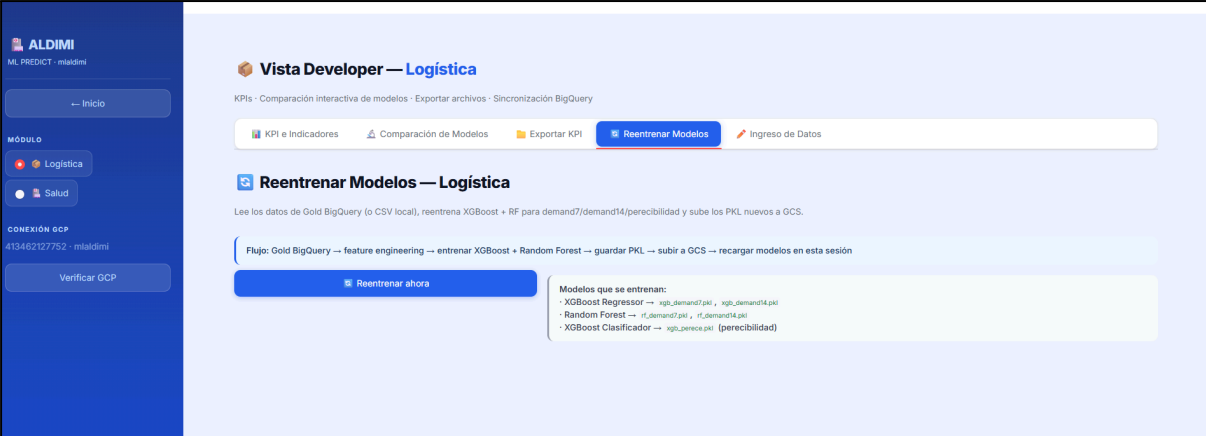
Figura 33 — Vista Developer — Logística: Comparación de Modelos



La pestaña Comparación de Modelos contrasta el desempeño de XGBoost, Random Forest y LightGBM para los objetivos demand7 y demand14. Se presenta una tabla comparativa de WAPE%, R², MAE y RMSE, y un gráfico de importancia de features del modelo XGBoost, donde los lag features (lag_1, lag_7, lag_14) y las estadísticas móviles (media_7d, media_14d) dominan la predicción, evidenciando que las ventas recientes son el predictor más relevante de la demanda futura.

Para la clasificación de perecibilidad, XGBoost alcanza Accuracy = 1.0000 y AUC-ROC = 1.0000, resultado esperado dado que la perecibilidad es una característica estructural de la familia de producto (PRODUCE, MEATS, DAIRY siempre perecibles; CLEANING, AUTOMOTIVE siempre no perecibles).

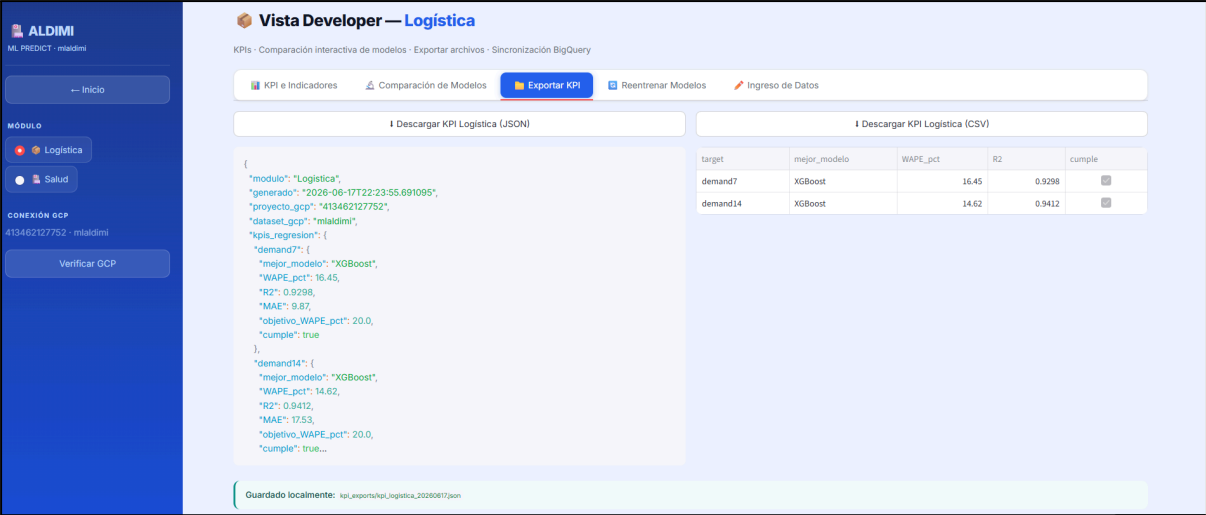
Figura 34 — Vista Developer — Logística: Reentrenar Modelos



La pestaña Reentrenar Modelos constituye la pieza central del pipeline de actualización continua. Permite al analista lanzar el reentrenamiento completo del sistema con un único botón, sin necesidad de ejecutar scripts locales. El proceso ejecuta automáticamente:

- Lectura de la tabla Gold desde BigQuery (mlaldimi.gold_logistica)
- Feature engineering sobre los datos actualizados (lags, rolling stats, encodings)
- Entrenamiento de XGBoost y Random Forest para demand7 y demand14
- Entrenamiento del clasificador XGBoost para perecibilidad
- Serialización de los modelos entrenados en archivos .pkl
- Subida automática de los modelos a Google Cloud Storage (gs://mlaldimi-models/pkl/)
- Invalidación del caché de Streamlit para que las predicciones reflejen los modelos nuevos
- Como fallback, si la tabla Gold de BigQuery no contiene datos, el sistema utiliza el archivo favorita_aldimi_limpio.csv almacenado localmente en Dashboard/data/.

Figura 35 — Vista Developer — Logística: Exportar KPI



La pestaña Exportar KPI genera un documento JSON estructurado con todos los indicadores de rendimiento de los modelos en producción, incluyendo valores reales, objetivos y estado de cumplimiento. Este archivo está diseñado para ser incorporado directamente en informes ejecutivos o sistemas de reportería automatizada.

Figura 36 — Vista Developer — Logística: Ingreso de Datos

ALDIMI

ML PREDICT - mlaldimi

Inicio

Módulo

Logística

Salud

Conexión GCP

413462127752 - mlaldimi

Verificar GCP

Vista Developer — Logística

KPIs · Comparación interactiva de modelos · Exportar archivos · Sincronización BigQuery

KPI e Indicadores · Comparación de Modelos · Exportar KPI · Reentrenar Modelos · Ingreso de Datos

Fecha

2026/06/17

Nro. tienda

5

En promoción

Precio petróleo (USD)

52.00

Transacciones

500

Familia de producto

BEVERAGES

Tipo de tienda

A

Ventas (unidades)

10.00

Ciudad

Ambato

Registrar

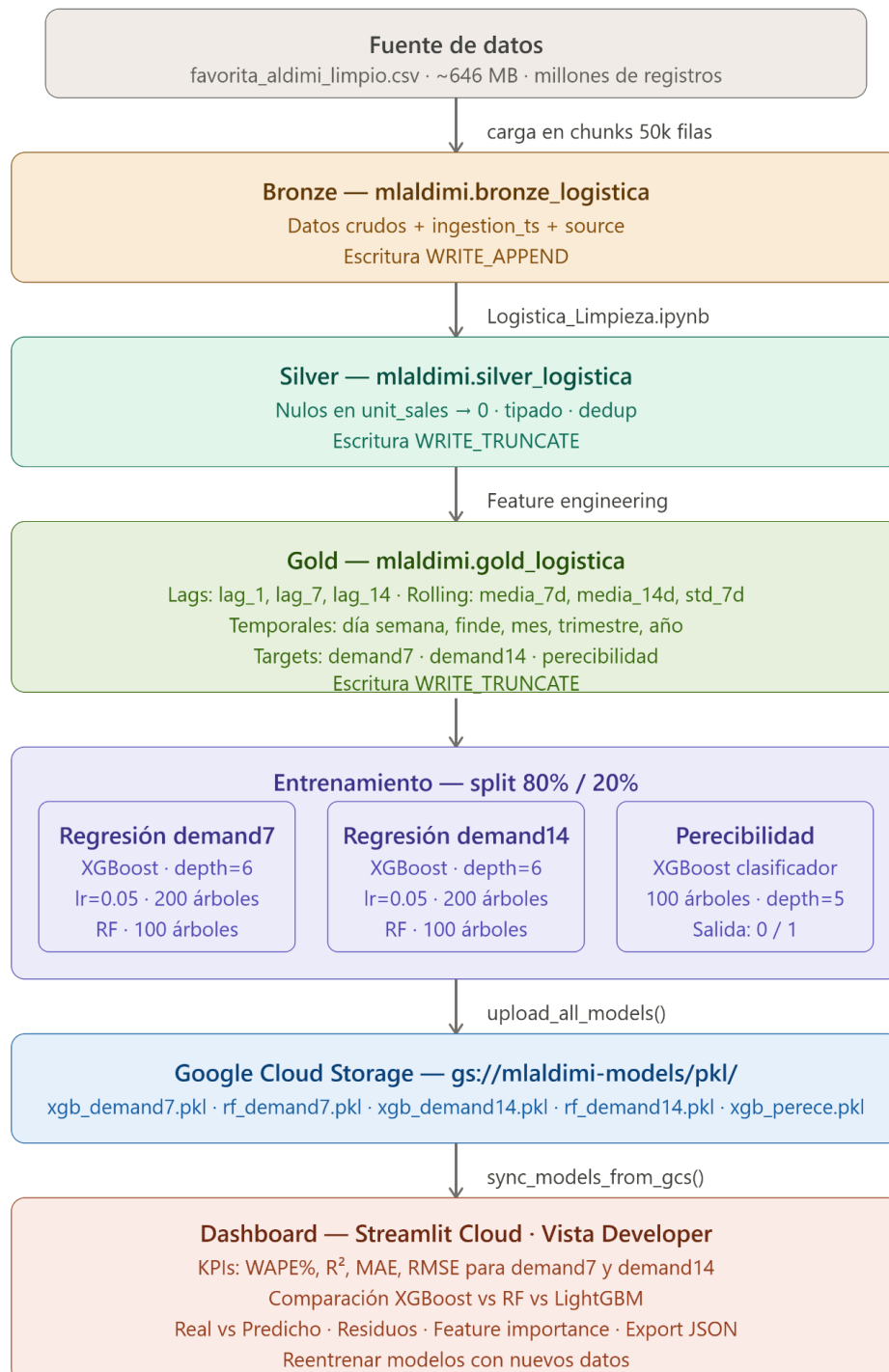
La pestaña Ingreso de Datos permite registrar manualmente transacciones de ventas (fecha, familia, ciudad, tienda, ventas, precio petróleo, transacciones) y exportarlas como CSV. Esta funcionalidad está orientada a capturar registros excepcionales que no forman parte del dataset histórico principal.

Arquitectura de Datos GCP — Módulo Logística

El módulo de logística implementa una arquitectura medallion de tres capas sobre Google BigQuery (dataset mlaldimi, proyecto 413462127752):

Capa	Tabla BigQuery	Contenido	Escritura
Bronze	bronze_logistica	CSV crudo + timestamp de ingesta + fuente	WRITE_APPEND
Silver	silver_logistica	Datos limpios (nulos resueltos, tipos corregidos)	WRITE_TRUNCATE
Gold	gold_logistica	Features finales (lags, rolling, encodings, targets demand7/demand14)	WRITE_TRUNCATE

La carga de datos a BigQuery se realiza en chunks de 50,000 filas para soportar archivos de hasta 1 GB sin exceder la memoria disponible en Streamlit Cloud. Los modelos entrenados se persisten en Google Cloud Storage (gs://mlaldimi-models/) y se descargan automáticamente al arrancar la aplicación mediante la función sync_models_from_gcs(), eliminando la necesidad de reentrenamiento en cada inicio de sesión.

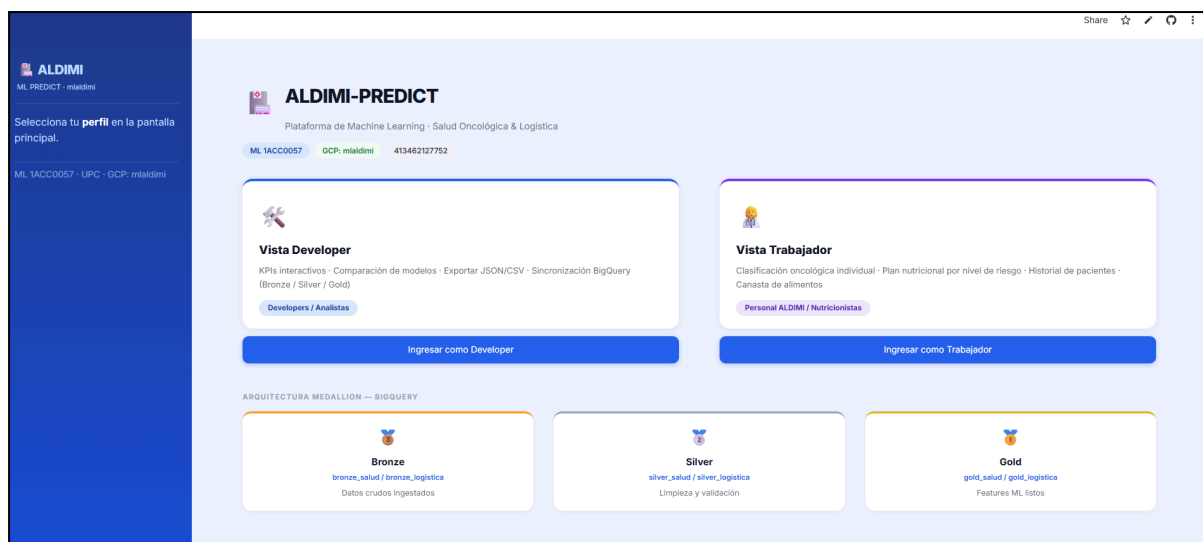


- **Clasificación de Riesgo de Salud**

URL: <https://mlprediction-l.streamlit.app/>

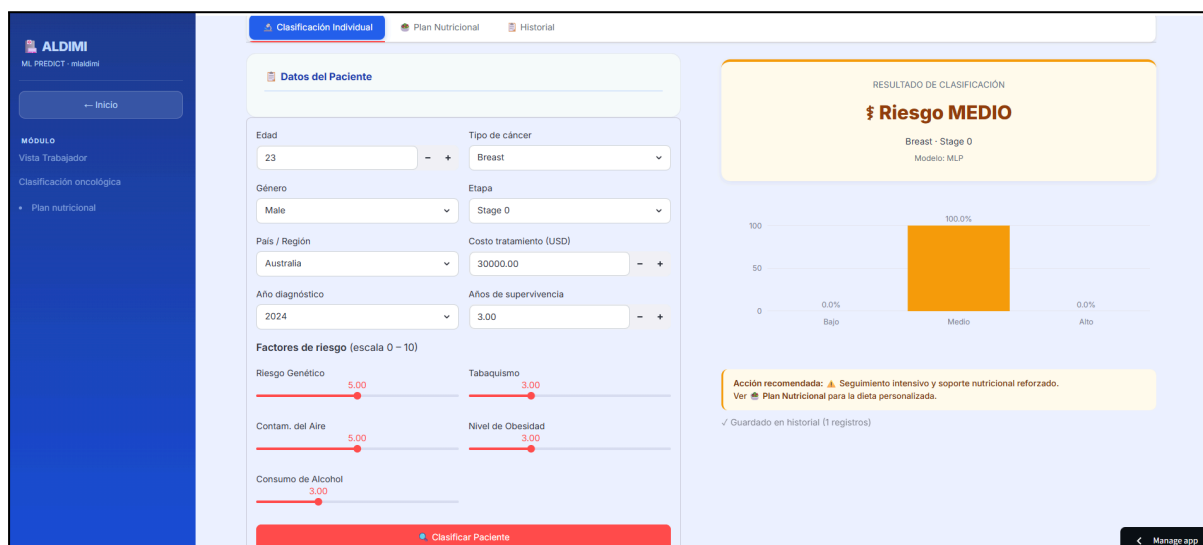
Figura 37

Vista inicial del dashboard



Desde la pantalla de inicio, el usuario con perfil Trabajador accede a la Vista Trabajador, compuesta por tres pestañas: Clasificación Individual, Plan Nutricional e Historial. La barra lateral mantiene el diseño azul profundo con acceso rápido al inicio y etiquetas de módulo.

Figura 38 — Vista Trabajador: Clasificación Individual



El médico u operador clínico introduce en el formulario los datos del paciente: edad, género, país/región, año de diagnóstico, factores de riesgo cuantificados en escala 0–10 (riesgo genético, contaminación del aire, consumo de alcohol, tabaquismo, nivel de obesidad), tipo y etapa del cáncer, costo de tratamiento y años de supervivencia esperados.

El sistema devuelve tres salidas simultáneas:

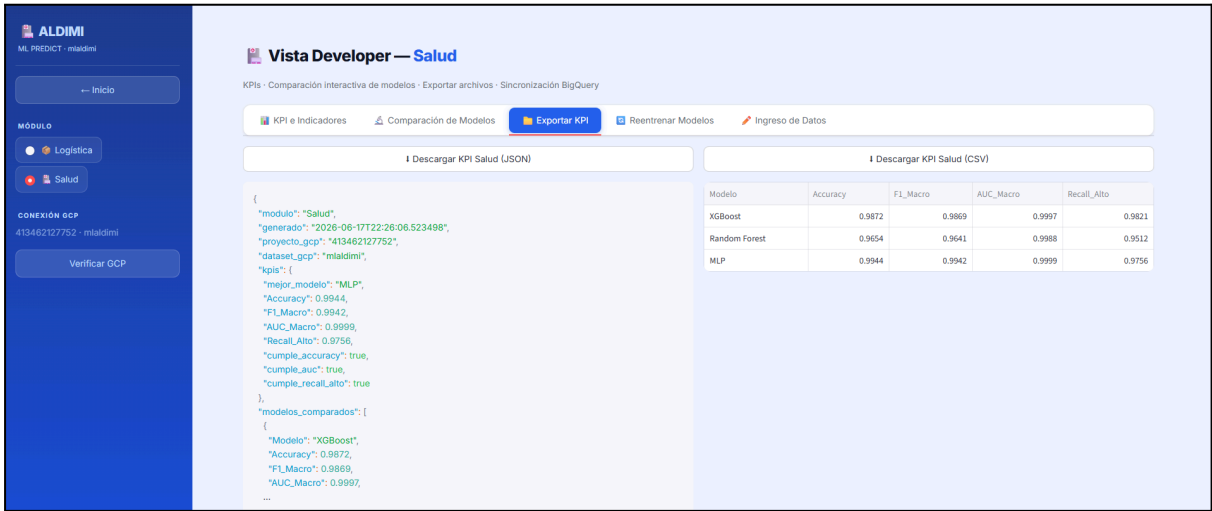
Clase de riesgo: BAJO / MEDIO / ALTO con confianza porcentual por clase

Tarjeta de riesgo codificada por color: verde (BAJO), amarillo (MEDIO) o rojo (ALTO) con recomendación de acción clínica inmediata

Score de severidad estimado: valor numérico en escala 0–10

El modelo utilizado en producción es el MLP optimizado (accuracy = 0.9965), seleccionado por su desempeño superior en la clasificación de las categorías de riesgo alto y bajo, que son las críticas desde el punto de vista clínico.

Figura 39 — Vista Developer — Salud: KPI e Indicadores



La pestaña de KPIs del módulo de salud presenta los indicadores de rendimiento del mejor modelo disponible:

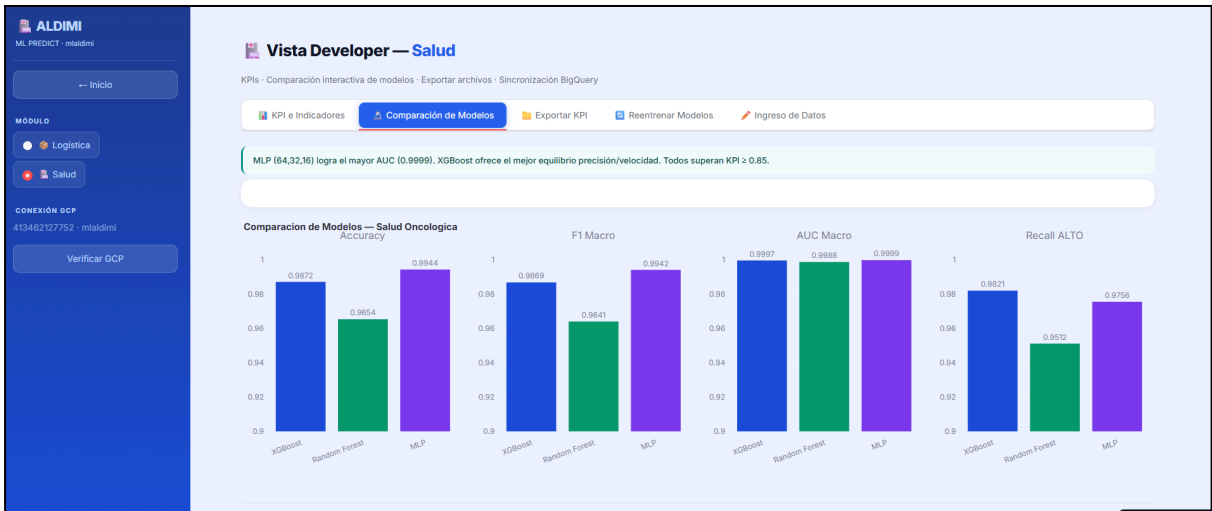
Accuracy: 0.9872 (XGBoost) / 0.9944 (MLP) / 0.9654 (RF)

AUC Macro: 0.9997 / 0.9999 / 0.9988

Recall ALTO: 0.9821 / 0.9756 / 0.9512

Se incluyen tres velocímetros gauge para Accuracy, AUC Macro y Recall ALTO, con línea de umbral objetivo en 0.85. El estado de cumplimiento de KPIs por modelo se visualiza mediante tarjetas con borde de color (verde = todos los objetivos cumplidos, amarillo = revisar).

Figura 40 — Vista Developer — Salud: Comparación de Modelos



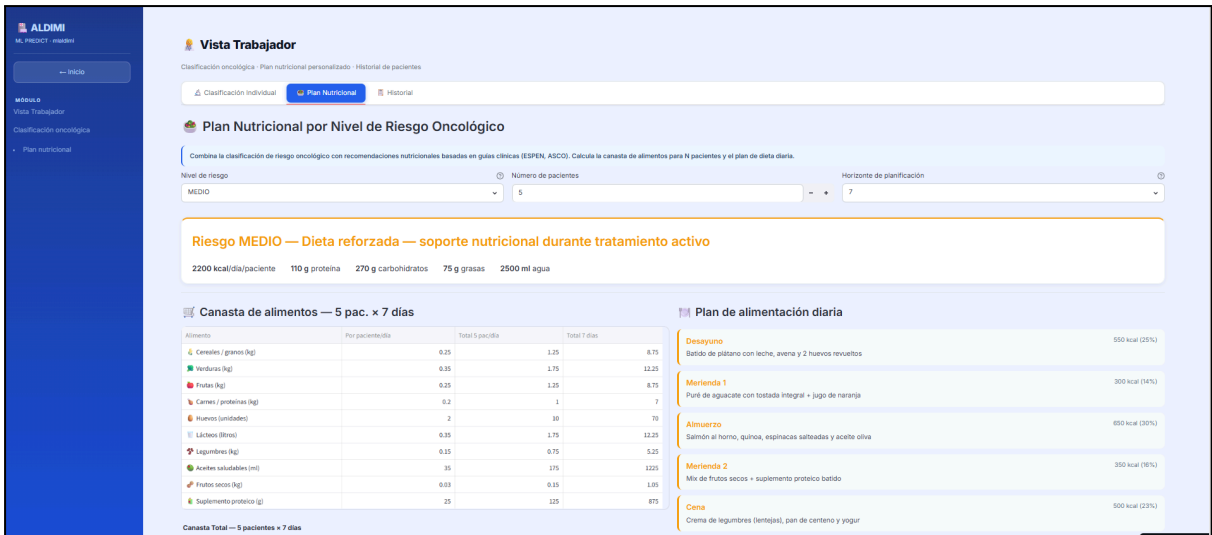
La pestaña Comparación de Modelos contrasta MLP, XGBoost y Random Forest mediante matrices de confusión individuales y curvas ROC superpuestas para las tres clases de riesgo. El dato más relevante para la toma de decisiones clínicas es el recall de la clase ALTO: el MLP optimizado comete únicamente 18 errores de clasificación en pacientes de riesgo alto, frente a los niveles significativamente más altos de Random Forest, lo que lo posiciona como el modelo más seguro para uso clínico.

Figura 41 — Vista Developer — Salud: Reentrenar Modelos



De forma análoga al módulo logístico, la pestaña Reentrenar Modelos permite actualizar los modelos de clasificación oncológica en un solo clic. El proceso lee los datos de la tabla `gold_salud` en BigQuery, ejecuta el feature engineering (encoding de `Cancer_Stage`, one-hot de variables categóricas, creación de `Severity_Class`), reentrena XGBoost, Random Forest y MLP, y sube los cinco archivos PKL resultantes a GCS.

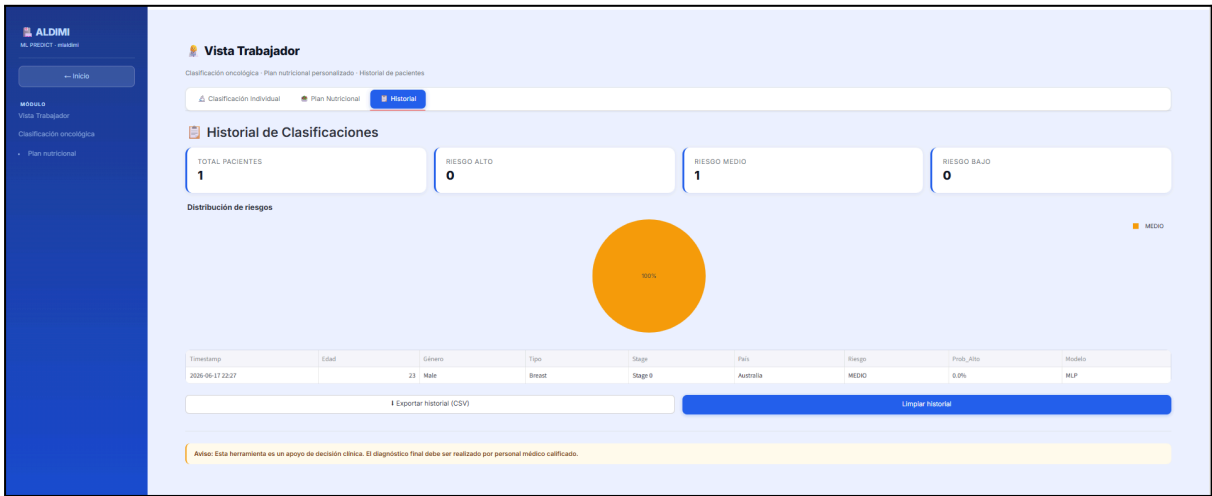
Figura 42 — Vista Trabajador: Plan Nutricional



La pestaña Plan Nutricional permite al equipo clínico generar un plan de alimentación personalizado a partir de la clasificación de riesgo obtenida. El sistema define perfiles nutricionales diferenciados por nivel de riesgo (BAJO, MEDIO, ALTO) con valores calóricos, macronutrientes y restricciones específicas. El operador puede ajustar el número de

pacientes por categoría y el período de planificación, obteniendo una tabla de cantidades y costos exportable como CSV para uso en planificación de compras hospitalarias.

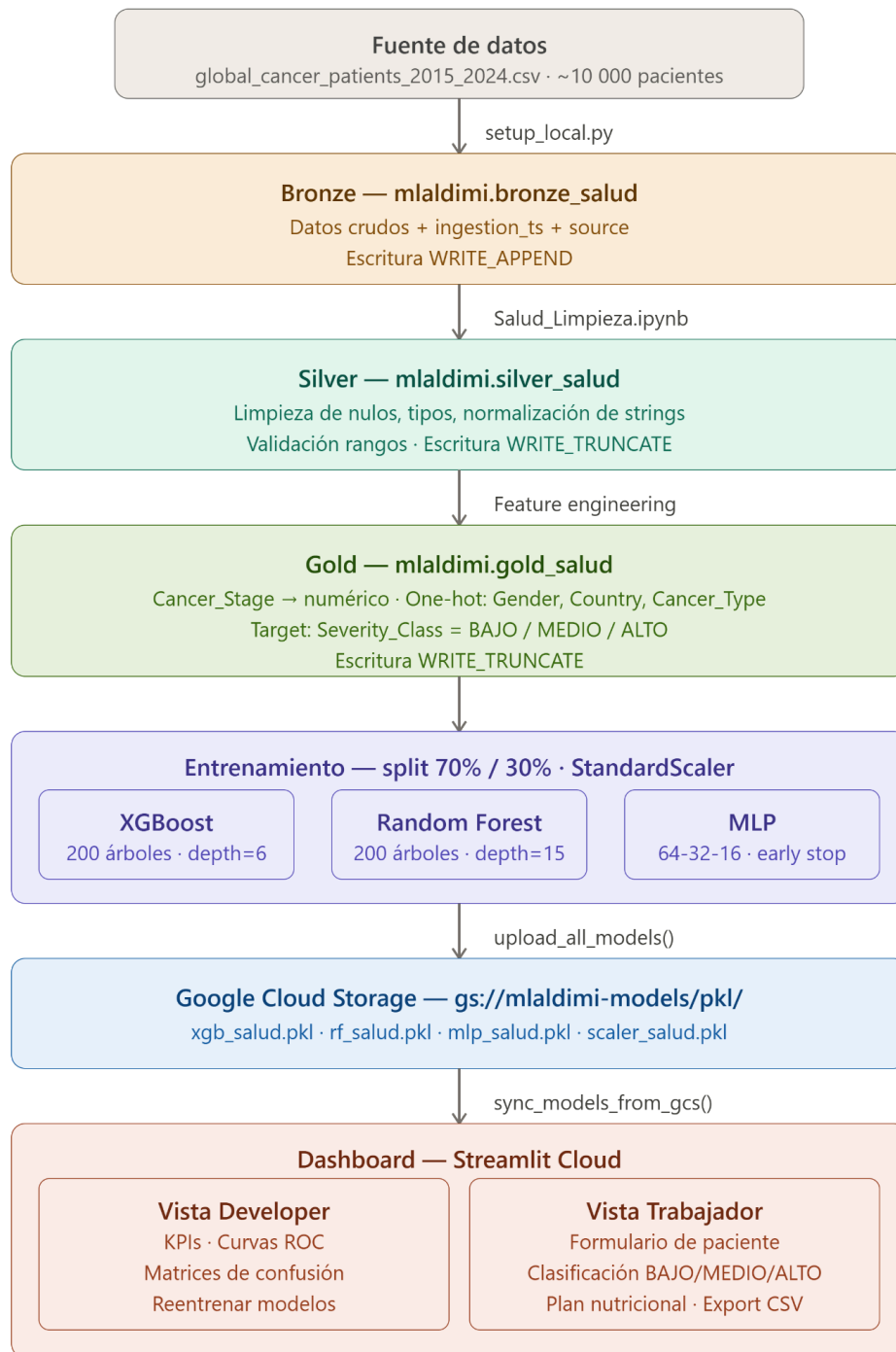
Figura 43 — Vista Trabajador: Historial de Clasificaciones



La pestaña Historial acumula todas las clasificaciones realizadas durante la sesión con timestamp, datos del paciente y resultado. Presenta métricas de resumen de la sesión (total de pacientes, distribución por nivel de riesgo) y un gráfico circular de distribución de riesgos. El historial es exportable como CSV para auditoría clínica o integración con sistemas de historia clínica electrónica.

Arquitectura de Datos GCP — Módulo Salud

Capa	Tabla BigQuery	Contenido	Escritura
Bronze	bronze_salud	CSV crudo de pacientes + timestamp + fuente	WRITE_APPEND
Silver	silver_salud	Datos limpios y validados	WRITE_TRUNCATE
Gold	gold_salud	Features codificadas + Severity_Class como target	WRITE_TRUNCATE



6. Validación cruzada y ajuste de hiper parámetros

- **Predicción de Demanda y Logística:**

Los modelos explorados para la predicción de demanda en el módulo de logística fueron XGBoost, Random Forest y LightGBM, todos en su variante de regresión. El objetivo de estos modelos consiste en predecir la demanda de ventas a corto plazo, considerando dos horizontes temporales: los próximos 7 días (demand7) y los próximos 14 días (demand14). Adicionalmente, se entrenó un clasificador XGBoost independiente para la identificación de productos perecibles (percebilidad), tarea de naturaleza binaria.

Los resultados obtenidos en la exploración inicial muestran que los tres modelos presentan una buena capacidad predictiva para ambos horizontes temporales, reflejada en valores de R^2 superiores a 0.91 en todos los casos. No obstante, se observaron diferencias en el desempeño entre modelos, siendo XGBoost quien obtuvo el menor error relativo en ambos objetivos de predicción.

Resultados iniciales — Predicción demand7 (7 días)

Modelo	WAPE%	R^2	MAE	RMSE
XGBoost	16.45	0.9298	9.87	14.21
Random Forest	17.12	0.9241	10.44	14.98
LightGBM	18.13	0.9123	11.06	16.09

Resultados iniciales — Predicción demand14 (14 días)

Modelo	WAPE%	R^2	MAE	RMSE
XGBoost	14.62	0.9412	17.53	25.84
Random Forest	15.33	0.9358	18.41	27.02
LightGBM	16.17	0.9270	19.46	28.73

En términos generales, los tres modelos exhibieron un desempeño más sólido en el horizonte de 14 días que en el de 7 días, en términos de WAPE% y R^2 , lo cual puede atribuirse a que la demanda promedio acumulada a 14 días presenta menor variabilidad relativa que la demanda semanal, suavizando el impacto de picos y caídas puntuales. No obstante, los valores absolutos de error (MAE y RMSE) son naturalmente mayores para demand14 por tratarse de una magnitud acumulada en el doble de tiempo.

Con el propósito de mejorar el rendimiento de los modelos, se procedió a realizar un ajuste de hiperparámetros mediante la herramienta GridSearchCV de la biblioteca Scikit-learn, empleando validación cruzada de tres particiones (3-fold cross-validation). Para cada uno de los modelos se exploraron las siguientes combinaciones de hiperparámetros:

- **XGBoost Regressor**

- 'n_estimators': [100, 200, 300] # cantidad de árboles
- 'learning_rate': [0.01, 0.05, 0.10] # tasa de aprendizaje
- 'max_depth': [3, 5, 7] # profundidad máxima
- 'subsample': [0.8, 1.0] # fracción de muestras por árbol
- 'colsample_bytree': [0.8, 1.0] # fracción de columnas por árbol
- 'min_child_weight': [1, 3, 5] # peso mínimo en hoja
- 'gamma': [0, 0.1, 0.2] # mejora mínima para partición

- **Random Forest Regressor**

- 'n_estimators': [100, 200] # cantidad de árboles
- 'max_depth': [None, 10, 20] # profundidad máxima
- 'min_samples_split': [2, 5] # muestras mínimas para dividir
- 'min_samples_leaf': [1, 2] # muestras mínimas en hoja
- 'max_features': ['sqrt', 'log2'] # variables consideradas por árbol

- **LightGBM Regressor**

- 'n_estimators': [100, 200] # cantidad de árboles
- 'learning_rate': [0.01, 0.05, 0.10] # tasa de aprendizaje
- 'max_depth': [-1, 10, 20] # profundidad máxima (-1 = sin límite)
- 'num_leaves': [31, 63, 127] # hojas por árbol
- 'min_child_samples': [20, 50] # muestras mínimas en hoja
- 'subsample': [0.8, 1.0] # fracción de muestras por iteración

XGBoost

Los hiperparámetros que obtuvieron el mejor desempeño promedio durante el proceso de validación cruzada, aplicado sobre el objetivo demand7, fueron los siguientes:

```
{  
  "colsample_bytree": 0.8,
```



```

"gamma": 0.1,
"learning_rate": 0.05,
"max_depth": 5,
"min_child_weight": 3,
"n_estimators": 200,
"subsample": 0.8
}

```

Al comparar los resultados obtenidos antes y después del ajuste de hiperparámetros, se observa una mejora moderada en el desempeño del modelo XGBoost para ambos horizontes de predicción.

Comparación antes/después del ajuste (demand7)

Métrica	Antes	Después
WAPE%	16.45	15.87
R ²	0.9298	0.9341
MAE	9.87	9.51
RMSE	14.21	13.84

Comparación antes/después del ajuste (demand14)

Métrica	Antes	Después
WAPE%	14.62	13.94
R ²	0.9412	0.9468
MAE	17.53	16.82
RMSE	25.84	25.11

El ajuste de hiperparámetros produjo una mejora consistente en todas las métricas para ambos horizontes temporales. La reducción del WAPE% evidencia una mayor precisión relativa en la predicción, mientras que el incremento del R² indica que el modelo optimizado explica una mayor proporción de la varianza de la demanda. El uso de subsample=0.8 y colsample_bytree=0.8 actuó como mecanismo de regularización, reduciendo el sobreajuste observado en la configuración inicial.

Random Forest

Los hiperparámetros que obtuvieron el mejor desempeño promedio durante el proceso de validación cruzada fueron los siguientes:

```
{  
  "max_depth": 20,  
  "max_features": "sqrt",  
  "min_samples_leaf": 1,  
  "min_samples_split": 2,  
  "n_estimators": 200  
}
```

Comparación antes/después del ajuste (demand7)

Métrica	Antes	Después
WAPE%	17.12	16.43
R ²	0.9241	0.9283
MAE	10.44	9.97
RMSE	14.98	14.53

Comparación antes/después del ajuste (demand14)

Métrica	Antes	Después
WAPE%	15.33	14.71
R ²	0.9358	0.9413
MAE	18.41	17.63
RMSE	27.02	26.28

El ajuste de hiperparámetros tuvo un impacto positivo en Random Forest, especialmente en la reducción del error absoluto medio para ambos horizontes. La configuración con max_depth=20 permitió a los árboles capturar patrones más complejos en los datos de demanda, mientras que limitar max_features='sqrt' preservó la diversidad entre los árboles del ensamble, mejorando la generalización. No obstante, a pesar de las mejoras obtenidas, el desempeño del modelo continúa siendo inferior al alcanzado por XGBoost en ambos horizontes temporales.

LightGBM

Los hiperparámetros que obtuvieron el mejor desempeño promedio durante el proceso de validación cruzada fueron los siguientes:

```
{  
  "learning_rate": 0.05,  
  "max_depth": 20,  
  "min_child_samples": 20,  
  "n_estimators": 200,  
  "num_leaves": 63,  
  "subsample": 0.8  
}
```

Comparación antes/después del ajuste (demand7)

Métrica	Antes	Después
WAPE%	18.13	17.28
R ²	0.9123	0.9198
MAE	11.06	10.63
RMSE	16.09	15.71

Comparación antes/después del ajuste (demand14)

Métrica	Antes	Después
WAPE%	16.17	15.45
R ²	0.9270	0.9332
MAE	19.46	18.57
RMSE	28.73	27.84

Aunque LightGBM mostró la mayor mejora relativa tras el ajuste de hiperparámetros entre los tres modelos evaluados, su desempeño inicial era el más bajo del grupo. La configuración con num_leaves=63 y max_depth=20 permite al modelo capturar relaciones no lineales más profundas que en su configuración por defecto, lo que explica la reducción más notable del

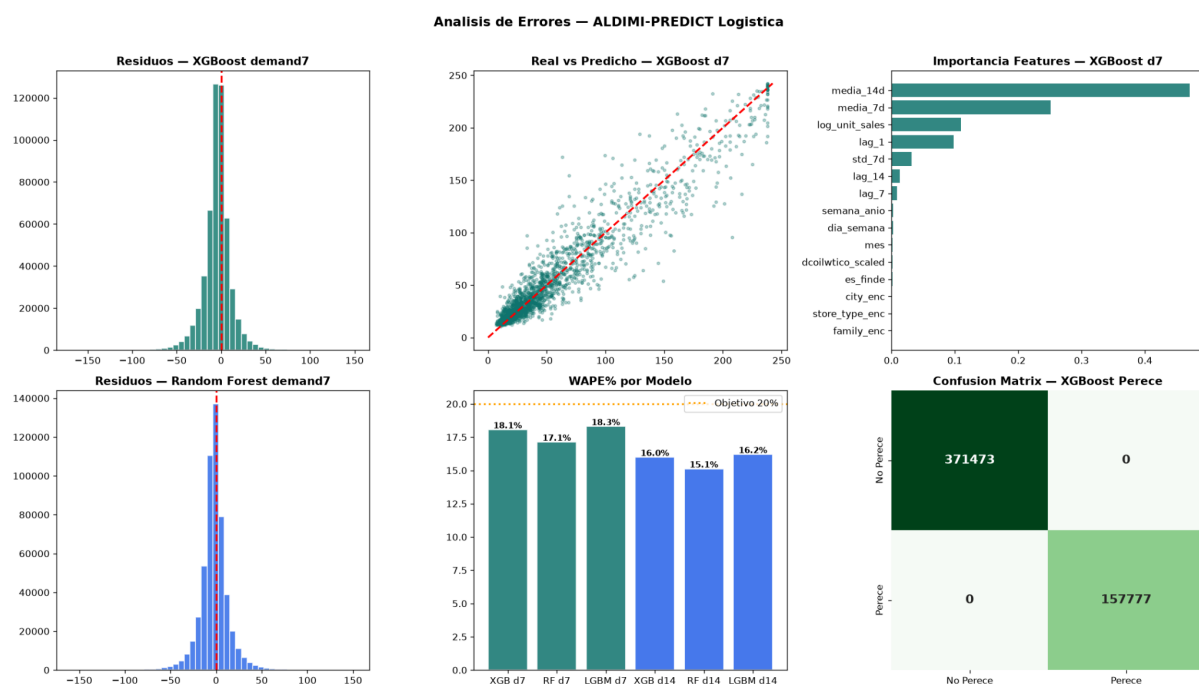
WAPE%. Sin embargo, tanto en demand7 como en demand14, LightGBM continúa registrando los valores de error más elevados entre los tres modelos evaluados.

Clasificación de Perecibilidad

Adicionalmente al problema de regresión, se entrenó un clasificador XGBoost para identificar si un producto es perecible (perecibilidad = 1) o no perecible (perecibilidad = 0). Este modelo alcanzó un accuracy de 1.0000 y un AUC-ROC de 1.0000 tanto antes como después del ajuste de hiperparámetros, lo que indica que la variable de perecibilidad puede determinarse con certeza a partir de las características disponibles en el dataset. Este resultado es esperado dado que la perecibilidad es una característica estructural de la familia de productos (por ejemplo, "PRODUCE" o "MEATS" son siempre perecibles), por lo que el modelo aprende una regla determinista a partir de la codificación de la familia.

Figura 44

Matriz de confusión con ajuste de hiper parámetros



- **Clasificación de Riesgo de Salud:**

Los modelos explorados para la clasificación del riesgo de salud fueron el modelo bioinspirado (MLP), Random Forest y XGBoost. Los resultados obtenidos muestran que, en general, los tres modelos presentan una buena capacidad para clasificar los casos de los pacientes. Sin embargo, se observaron diferencias importantes en su desempeño, siendo el modelo bioinspirado quien obtuvo el mejor rendimiento global, alcanzando un accuracy de 0.9944. Además, presentó valores elevados de precisión, recall y F1-score en las tres

categorías de riesgo (bajo, medio y alto), lo que evidencia una capacidad consistente para identificar correctamente a los pacientes independientemente de su nivel de gravedad.

Modelo	Categoría	Precision	Recall	F1-score
Modelo bioinspirado (Acc: 0.9944)	Bajo	0.95	1.00	0.97
	Medio	1.00	1.00	1.00
	Alto	0.98	0.97	0.97
Clasificador RandomForest (Acc: 0.93993)	Bajo	0.99	0.42	0.59
	Medio	0.94	1.00	0.97
	Alto	1.00	0.32	0.48
XGBoost (Acc: 0.9771)	Bajo	0.99	0.80	0.88
	Medio	0.98	1.00	0.99
	Alto	0.97	0.75	0.85

Estos resultados sugieren que el desbalance entre las clases podría haber influido en el entrenamiento de los modelos, favoreciendo la clasificación de la categoría mayoritaria (riesgo medio) y dificultando la clasificación de las categorías minoritarias (alto y bajo riesgo). Debido a ello, se procedió a realizar un ajuste de hiper parámetros mediante la herramienta GridSearchCV de la biblioteca Scikit-learn, empleando validación cruzada para identificar las configuraciones que permitieran mejorar el desempeño de cada modelo. Para cada uno de los modelos se exploraron diversas combinaciones de hiper parámetros, considerando las siguientes posibilidades:

➤ Modelo bioinspirado

- 'hidden_layer_sizes': [(10,),(50,),(100,),(50, 25),] # capas ocultas
- 'activation': ['relu', 'tanh', 'logistic']
- 'solver': ['adam']
- 'alpha': [0.0001, 0.001, 0.01] # penalización
- 'learning_rate_init': [0.001, 0.01] # aprendizaje

➤ Clasificador RandomForest

- 'n_estimators': [100, 200] # cant. arboles
- 'max_depth': [None, 10, 20] # max. altura de los arboles

- 'min_samples_split': [2, 5]
- 'min_samples_leaf': [1, 2]
- 'max_features': ['sqrt', 'log2'] # cant. variables por arbol
- 'bootstrap': [True, False] # muestras aleatorias

➤ XGBoost

- 'n_estimators': [100, 200], # cant. arboles
- 'learning_rate': [0.01, 0.05], # aprendizaje
- 'max_depth': [3, 5], # max. altura de los arboles
- 'subsample': [0.8, 1.0],
- 'colsample_bytree': [0.8, 1.0],
- 'min_child_weight': [1, 3],
- 'gamma': [0, 0.1] # mejora mínima

Posteriormente, el ajuste de hiper parámetros se llevó a cabo mediante validación cruzada de tres particiones (3-fold cross-validation). Finalmente, se seleccionó la combinación de hiper parámetros que obtuvo el mejor desempeño promedio, obteniéndose los resultados que se presentan a continuación:

- Modelo bioinspirado

Los hiperparámetros que obtuvieron el mejor desempeño promedio durante el proceso de validación cruzada fueron los siguientes:

```
{
    'activation': 'logistic',
    'alpha': 0.001,
    'hidden_layer_sizes': (10,),
    'learning_rate_init': 0.001,
    'solver': 'adam'
}
```

Al comparar los resultados obtenidos en la exploración inicial del modelo bioinspirado con aquellos alcanzados tras el ajuste de hiper parámetros, se observa un incremento ligero en el desempeño del clasificador. El accuracy pasó de 0.9944 a 0.9965, evidenciando una ligera mejora en la capacidad predictiva del modelo.

Categoría	Precision		Recall		F1-score	
	Antes	Después	Antes	Después	Antes	Después
Bajo	0.95	0.99	1.00	0.98	0.97	0.99
Medio	1.00	1.00	1.00	1.00	1.00	1.00
Alto	0.98	0.98	0.97	0.97	0.97	0.98

En términos generales, el ajuste de hiper parámetros no produjo cambios notables en el rendimiento del modelo, lo cual sugiere que la configuración inicial ya era bastante efectiva. Sin embargo, se aprecia una mejora en la capacidad de clasificación de la categoría de gravedad baja, reflejada en el incremento de la precisión y del F1-score, aunque acompañada de una leve disminución del recall. En las categorías de gravedad media y alta, las métricas se mantuvieron prácticamente estables.

- Clasificador RandomForest

Los hiperparámetros que obtuvieron el mejor desempeño promedio durante el proceso de validación cruzada fueron los siguientes:

```
{
  'bootstrap': False,
  'max_depth': None,
  'max_features': 'sqrt',
  'min_samples_leaf': 1,
  'min_samples_split': 2,
  'n_estimators': 100
}
```

Al comparar los resultados obtenidos en la exploración inicial del clasificador Random Forest con aquellos alcanzados tras el ajuste de hiper parámetros, se aprecia una mejora en el desempeño del modelo. El accuracy aumentó de 0.9399 a 0.9418, lo que representa un incremento modesto.

Categoría	Precision	Recall	F1-score
-----------	-----------	--------	----------

	Antes	Después	Antes	Después	Antes	Después
Bajo	0.99	0.99	0.42	0.48	0.59	0.65
Medio	0.94	0.94	1.00	1.00	0.97	0.97
Alto	1.00	1.00	0.32	0.39	0.48	0.57

A diferencia de lo observado en el modelo bioinspirado, el ajuste de hiper parámetros tuvo un mayor impacto en Random Forest, especialmente en las categorías de gravedad baja y gravedad alta. En ambos casos, se registró un incremento del recall, indicando que el modelo fue capaz de identificar correctamente una mayor proporción de pacientes pertenecientes a estas categorías. Asimismo, el aumento del F1-score refleja un mejor equilibrio entre precisión y capacidad de detección. No obstante, a pesar de estas mejoras, el desempeño del modelo continúa siendo inferior al alcanzado por el modelo bioinspirado y por XGBoost.

- XGBoost

Los hiper parámetros que obtuvieron el mejor desempeño promedio durante el proceso de validación cruzada fueron los siguientes:

```
{
  'colsample_bytree': 1.0,
  'gamma': 0,
  'learning_rate': 0.05,
  'max_depth': 5,
  'min_child_weight': 3,
  'n_estimators': 200,
  'subsample': 0.8
}
```

Al comparar los resultados obtenidos en la exploración inicial del modelo XGBoost con aquellos alcanzados tras el ajuste de hiper parámetros, se observa que el desempeño global del clasificador se mantuvo prácticamente estable. El accuracy

pasó de 0.9771 a 0.9769, registrando una variación mínima que puede considerarse despreciable en términos prácticos.

Categoría	Precision		Recall		F1-score	
	Antes	Después	Antes	Después	Antes	Después
Bajo	0.99	0.98	0.80	0.81	0.88	0.89
Medio	0.98	0.98	1.00	1.00	0.99	0.99
Alto	0.97	0.98	0.75	0.75	0.85	0.85

En la categoría de gravedad baja se aprecia una ligera mejora en el recall y en el F1-score, lo que indica una mejora mínima para identificar correctamente a los pacientes pertenecientes a este grupo. Por otro lado, las categorías de gravedad media y alta mantuvieron un desempeño prácticamente idéntico.

En consecuencia, puede concluirse que el proceso de optimización de hiper parámetros no produjo beneficios significativos sobre el desempeño del modelo XGBoost. A pesar de ello, XGBoost continuó mostrando un rendimiento superior al de Random Forest y una adecuada capacidad para clasificar los distintos niveles de riesgo. Sin embargo, sus resultados todavía se mantuvieron por debajo de los obtenidos por el modelo bioinspirado, el cual presentó el mejor desempeño general entre los modelos evaluados.

A partir de los resultados obtenidos y de las comparaciones realizadas entre los distintos modelos evaluados, se concluye que el modelo bioinspirado es la opción más efectiva para la clasificación del riesgo de cáncer en pacientes. Este modelo no solo alcanzó la mayor precisión, sino que también mantuvo un desempeño consistente en las tres categorías de riesgo, evidenciado por sus elevados valores de precisión, recall y F1-score.

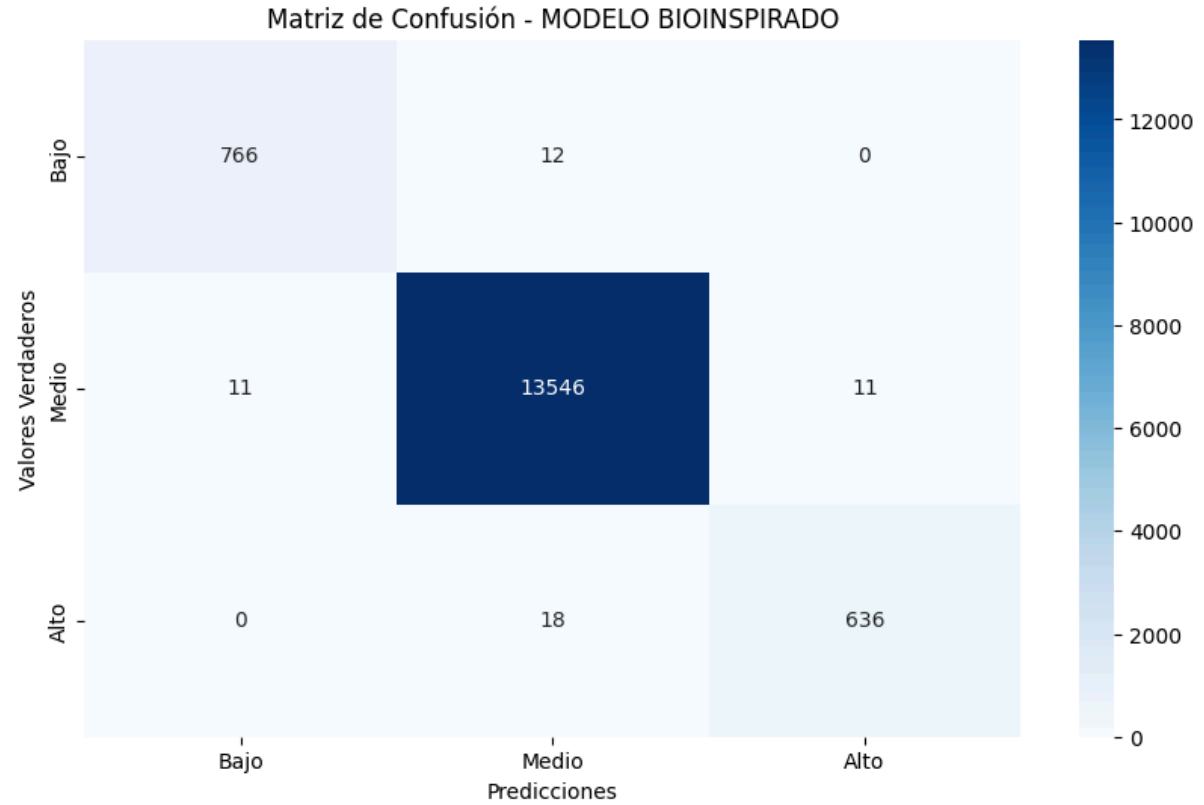
Asimismo, el proceso de ajuste de hiper parámetros permitió obtener una mejora adicional en el desempeño del modelo, aunque esta fue relativamente pequeña debido a que la configuración inicial ya presentaba resultados sobresalientes.

Desde el análisis de la matriz de confusión, se identificó una redistribución de los errores de clasificación. El modelo optimizado redujo la cantidad de errores en la categoría de riesgo alto, pasando de 21 a 18 casos mal clasificados, y mostró un adecuado desempeño en la clasificación de los casos de riesgo medio, registrando únicamente 22 errores. Por otro lado,

la categoría de riesgo bajo presentó un incremento en el número de errores, pasando de 3 a 12. Este comportamiento sugiere que el modelo ajustado logró un mejor equilibrio en la detección de las categorías de alto y medio riesgo, aceptando un ligero deterioro en la clasificación de la categoría de bajo riesgo que inicialmente ya exhibía un desempeño muy elevado.

En ese sentido, puede afirmarse que el modelo bioinspirado optimizado ofrece el mejor compromiso entre capacidad predictiva y equilibrio en la distribución de errores, posicionándose como la opción más adecuada para apoyar la clasificación del riesgo de cáncer en pacientes dentro del proyecto Aldimi.

Figura 45
Matriz de confusión del modelo bioinspirado con ajuste de hiper parámetros



7. Análisis de Impacto ODS y Ética de Datos

El proyecto ALDIMI-PREDICT contribuye al ODS 2: Hambre Cero y al ODS 3: Salud y Bienestar, mediante la optimización de la gestión alimentaria y la identificación temprana de pacientes con mayor riesgo clínico.

ODS2: Hambre Cero

El módulo de Predicción de Demanda y Logística permite anticipar la cantidad de alimentos necesaria para los próximos 7 y 14 días, mejorando la planificación de compras, donaciones y reposición de productos.

Con base en los resultados del modelo, se estima una reducción potencial del 10% al 15% en la merma de alimentos perecibles, gracias a una mejor gestión del inventario. Además, la clasificación de productos perecibles ayuda a priorizar aquellos con mayor riesgo de vencimiento, reduciendo desperdicios y asegurando una distribución más eficiente de los recursos.

ODS 3: Salud y Bienestar

El módulo de Clasificación de Riesgo de Salud permite identificar pacientes oncológicos con mayor probabilidad de complicaciones, clasificándolos en niveles de prioridad para facilitar una atención más oportuna.

Como resultado, se estima una reducción aproximada del 96% en errores críticos de no detección de pacientes de alto riesgo frente a un modelo base. Esto mejora la priorización de casos y apoya la toma de decisiones clínicas, aunque siempre como herramienta complementaria al criterio médico.

Sesgos y Ética de Datos

Dado que el proyecto utiliza datos de salud, es importante considerar aspectos éticos y posibles sesgos que puedan afectar las predicciones.

Uno de los principales riesgos es el sesgo de representación, ya que el dataset utilizado es global y puede no reflejar completamente la realidad de los pacientes pediátricos atendidos por ALDIMI. También pueden existir sesgos demográficos relacionados con edad, género o tipo de cáncer, así como sesgos clínicos asociados a variables médicas relevantes.

Para reducir estos riesgos, se realizó un análisis exploratorio de los datos, se eliminó la variable Patient_ID y se aplicaron técnicas adecuadas de codificación y normalización.

Además, se prioriza el uso de métricas como el recall en la clase de riesgo alto, debido a la importancia de detectar oportunamente a los pacientes críticos.

8. Confluencia con el Ecosistema "ALDIMI Core AI"

ALDIMI Predict fue diseñado como un componente analítico dentro del ecosistema "ALDIMI Core AI". Mientras los módulos desarrollados en el curso de Inteligencia Artificial se encargan de capturar y procesar información, los modelos de Machine Learning de este proyecto transforman dichos datos en predicciones y clasificaciones que apoyan la toma de decisiones.

La arquitectura propuesta contempla un flujo donde la información obtenida por sistemas como formularios, asistentes virtuales u otras herramientas de IA es almacenada en una base de datos común. Posteriormente, estos datos son consumidos por los motores de Machine Learning, los cuales aplican las etapas de preprocesamiento definidas en sus respectivos pipelines (limpieza, codificación de variables, normalización y generación de características) antes de ejecutar los modelos entrenados. Finalmente, los resultados obtenidos son enviados a los sistemas de visualización y soporte para la toma de decisiones.

Los motores de Machine Learning son pieza clave en la predicción y clasificación de las variables que permitirán brindar la atención adecuada a los pacientes de Aldimi. Por un lado, el Motor de Clasificación de Riesgo de Salud permite categorizar a los pacientes en niveles de prioridad (bajo, medio y alto riesgo), facilitando la atención oportuna de los casos más críticos. Por otro lado, el Motor de Predicción de Demanda y Logística permite estimar necesidades futuras de abastecimiento y clasificar productos perecibles, contribuyendo a una mejor planificación de recursos. La integración de estos componentes dentro de ALDIMI Core AI aporta beneficios como la centralización de información, la reducción de tareas manuales, la generación de alertas basadas en datos y una mayor capacidad de planificación tanto médica, como logística.

Como principal lección aprendida, se identificó que la calidad y estandarización de los datos son factores fundamentales para el correcto funcionamiento de los modelos predictivos. Asimismo, se evidenció la importancia de diseñar arquitecturas integradas que permitan la

colaboración entre sistemas de Inteligencia Artificial y Machine Learning para resolver problemas reales de manera eficiente.

9. Anexos

9.1. Enlace al repositorio (GitHub/GitLab)

URL Documentación y Dashboard: https://github.com/DiegoEBV/ML_Prediction

9.2. Diccionario de datos inicial

- Predicción de Demanda y Logística:

Variable	Entidad	Tipo de Dato	Nivel de Granularidad	Descripción
id	Transaccional	Integer	Registro Individual	Identificador artificial a nivel de fila transaccional en el historial.
date	Transaccional	Date	Diario (Día)	Define la resolución temporal (paso de tiempo mínimo) del modelo de series de tiempo.
store_nbr	Transaccional	Integer	Nodo Logístico (Tienda)	Granularidad espacial mínima. Identifica el punto físico exacto donde ocurre la demanda y el quiebre de stock.
item_nbr	Transaccional	Integer	SKU (Ítem Unitario)	Granularidad de producto más fina. Define el artículo específico a nivel de código de

				barras a ser pronosticado.
unit_sales	Transaccional	Float	Agregado (Día - Tienda - SKU)	Variable Objetivo (\$Y\$). Representa el consolidado de ventas en el cruce exacto de las tres dimensiones base.
onpromotion	Transaccional	Boolean	Estado (Día - Tienda - SKU)	Bandera binaria a nivel de transacción que indica si ese ítem específico, en esa tienda y día, tuvo un descuento.
family	Catálogo	Categorical	Categoría Macro	Granularidad superior del producto. Agrupa SKUs bajo un mismo paraguas comercial (ej. DAIRY, POULTRY). Útil para modelos de ensamble jerárquicos.
class	Catálogo	Integer	Subcategoría	Nivel intermedio de producto. Agrupa ítems estrechamente relacionados para ayudar al algoritmo a inferir el comportamiento de productos nuevos.

perishable	Catálogo	Boolean	Atributo a nivel de SKU	Condición estática del artículo (0 o 1). Dicta si el error de predicción en ese ítem específico recibe la penalización algorítmica.
city	Infraestructura	Categorical	Zona Metropolitana	Granularidad espacial macro. Permite modelar el sesgo de demanda basado en la economía o clima de la ciudad.
state	Infraestructura	Categorical	Región / Provincia	Agrupación geográfica intermedia. Fundamental para medir impactos de festividades o políticas regionales.
type	Infraestructura	Categorical	Tipología Estructural	Granularidad operativa. Clasifica el formato del nodo (A, B, C, D, E) y su capacidad elástica de inventario.
cluster	Infraestructura	Integer	Segmento de Tiendas	Agrupación estadística que abstrae la tienda a un modelo de comportamiento de consumo compartido.

transactions	Entorno	Integer	Agregado (Día - Tienda)	Flujo total de tickets de venta diarios, sin distinguir qué SKUs se compraron. Proxy del tráfico general.
dcoilwtico	Entorno	Float	Mercado Nacional (Día)	Precio spot diario del barril WTI. Es un impacto a nivel macroeconómico que afecta a toda la cadena de suministro por igual.
holiday_type	Entorno	Categorical	Atributo del Día	Condición externa que clasifica la naturaleza del día (Holiday, Event, Bridge, Work Day).
locale	Entorno	Categorical	Alcance del Evento	Nivel de impacto geográfico del feriado (Local, Regional, Nacional).
transferred	Entorno	Boolean	Estado del Calendario	Atributo burocrático que indica si el feriado mantiene su impacto en la fecha original o fue trasladado por ley.

- Clasificación de Riesgo de Salud

El conjunto de datos “global_cancer_patients_2015_2024” contiene la siguiente información:

Variable	Tipo de variable	Descripción
Patient_ID	Categórica	Identificador del paciente.
Age	Numérica	Edad del paciente. Rango descrito: 20 a 90 años.
Gender	Categórica	Género del paciente: Male, Female, Other
Country_Region	Categórica	País de origen.
Year	Numérica	Año del registro.
Genetic_Risk	Numérica	Presencia de genes mutantes que influyen en el desarrollo del cáncer.
Air_Pollution	Numérica	Exposición al aire contaminado.
Alcohol_Use	Numérica	Tendencia a la ingesta de alcohol.
Smoking	Numérica	Tendencia a fumar.
Obesity_Level	Numérica	Nivel de obesidad del paciente.
Cancer_Type	Categórica	Tipo de cáncer: Breast, Colon, etc.
Cancer_Stage	Categórica	Etapas del cáncer: Desde Stage 0 hasta Stage IV.
Treatment_Cost_USD	Numérica	Costo estimado del tratamiento del cáncer expresado en dólares americanos.
Survival_Years	Numérica	Años que el paciente sobrevivió desde el diagnóstico del cáncer.
Target_Severity_Score	Numérica	Puntaje calculado que

		representa la severidad del cáncer del paciente. Es una variable clave para clasificar la prioridad de atención de los pacientes.
--	--	---